ADVENTIST UNIVERSITY OF CENTRAL AFRICA (AUCA)

Advanced Big Data Analytics

Project I: Smart Energy Grid Analytics

Group member	Student ID	Role
BIZIMUNGU Theoneste	101398	Database and ingestion lead
NIYITANGA Eric	101393	Analytics, forecasting and repository lead
NDASHIMYE Janvier	101363	Storage, aggregates and dashboards lead

GitHub repository: https://github.com/ericniyitanga082/Big-Data-Analytics-Project1

Table of Contents

Executive summary **3**

1 Build the ingestion pipeline **4**

1.1 Confirm the environment 4

1.2 Create the database structures 4

1.3 Capture and publish a pilot stream 6

2 Capture and prepare the historical dataset **7**

2.1 Capture and load the complete stream 7

2.2 Investigate data quality 8

2.3 Clean and preserve a consistent benchmark 9

3 Compare chunk intervals **10**

3.1 Create and verify three configurations 10

3.2 Implement and time the five workloads 12

3.3 Explain the chunk results 16

4 Compare compression strategies **17**

4.1 Create the two compressed configurations 17

4.2 Measure storage and compression cost 19

4.3 Repeat the query comparison 20

4.4 Select a storage configuration 22

5 Investigate the utility data **23**

5.1 Identify demand patterns 23

5.2 Compare regions and customer types 24

5.3 Assess reporting quality 25

5.4 Investigate abnormal consumption 26

5.5 Management recommendation 28

6 Create and investigate continuous aggregates **28**

6.1 Create the three aggregate views 29

6.2 Test freshness and delayed arrivals 30

7 Build and compare dashboards **31**

7.1 Build the Raw dashboard 31

7.2 Build the Aggregate dashboard 33

7.3 Verify matching results 35

7.4 Measure performance and judge the tradeoff 36

8 Forecast the next day **36**

8.1 Prepare chronological data and features 36

8.2 Evaluate the previous-day baseline 38

8.3 Train and evaluate one regression model 38

8.4 Judge forecast usefulness 39

9 Reproducibility and contributions **41**

9.1 Report organisation 41

9.2 Make the work reproducible 41

9.3 Document contributions 42

Appendix A: Supporting SQL queries **43**

Appendix B: per-chunk compression durations **46**

Appendix C: derived calculations **46**

Appendix D: limitations and threats to validity **47**

Appendix E: glossary **48**

Appendix F: index of figures **49**

Executive summary

This report documents the design, execution and evaluation of a smart-meter analytics platform for 4,000 meters over 28 days (1 to 28 September 2026, 15-minute readings). The platform ingests readings through an MQTT broker and a PostgreSQL database with TimescaleDB, cleans and audits the data, compares storage layouts, builds pre-aggregated views and dashboards, and forecasts hourly regional consumption one day ahead.

Key results

Area	Result	Section
Historical load	10,696,848 readings emitted, consumed and committed; all three counts agree. 197.52 s at 54,156.84 rows/s.	2.1
Data quality	Completeness 98.993%; missing 1.007%, duplicates 0.499%, delayed 2.002%. All counts equal the reference table.	2.2
Cleaning	53,113 identical duplicates removed; 10,643,735 rows remain; 0 conflicting pairs.	2.3
Chunk intervals	No interval wins every query. 1 week is fastest on Q3, Q4 and Q5; 3 hours on Q1; 1 day ties on Q2. 3-hour Q3 is 71 times slower than 1 week.	3.2, 3.3
Compression	1,423.87 MB falls to 617.10 MB with meter segmentation (2.31×, 56.66% saved) and 619.57 MB without (2.30×).	4.2
Meter segmentation	Single-meter query 2.12 ms against 26.82 ms (12.7× faster); network-wide Q3 43.5% slower; compression about 1.9× longer.	4.4
Demand pattern	Highest demand 7,993.69 kW on Monday 28 September at 13:00. Weekend days use about 111,700 to 119,000 kWh against 141,000 to 152,500 kWh on weekdays.	5.1
Customer mix	Industrial meters are 10.15% of meters and use 57.62% of energy.	5.2
Anomaly rule	The fixed-threshold rule detected 3 of 40 events with 80,109 false flags; a per-meter rule is recommended.	5.4
Aggregates	Three continuous aggregates populated (26,880, 6,720 and 112,000 rows) in 71.78 s of manual refresh.	6.1
Freshness test	All four stages match the expected transition table; raw table holds 3 rows and 3.5 kWh.	6.2
Forecast	LightGBM test MAE 8.85 kWh against 42.92 kWh for the previous-day baseline (79.39% lower); better in all ten regions.	8.3, 8.4

Section 9.2 lists the repository and the execution order, and Appendix A contains the supporting SQL queries. Where a result could not be fully evidenced, the relevant subsection says so, and Appendix D collects the limitations.

1 Build the ingestion pipeline

1.1 Confirm the environment

The ingestion stack runs in Docker containers on a single Windows machine. EMQX 5.3.0 is the MQTT broker, and PostgreSQL 15 with TimescaleDB runs in the container started from the image timescale/timescaledb:latest-pg15. The container maps host port 5434 to PostgreSQL port 5432. All project tables live in one database, **auca_energy_db**. **PostgreSQL therefore runs in a container**, not natively and not in a separate virtual machine.

Item	Value	Source
MQTT broker	EMQX 5.3.0 (image emqx/emqx:5.3.0), ports 1883 (MQTT) and 18083 (dashboard)	Figure 1.1-a
Database server	PostgreSQL 15 with TimescaleDB, image timescale/timescaledb:latest-pg15, host port 5434	Figure 1.1-a
Project database	auca_energy_db	psql prompt in Figure 1.2-a
PostgreSQL runtime	Docker container (Docker Desktop on the host)	Figure 1.1-a

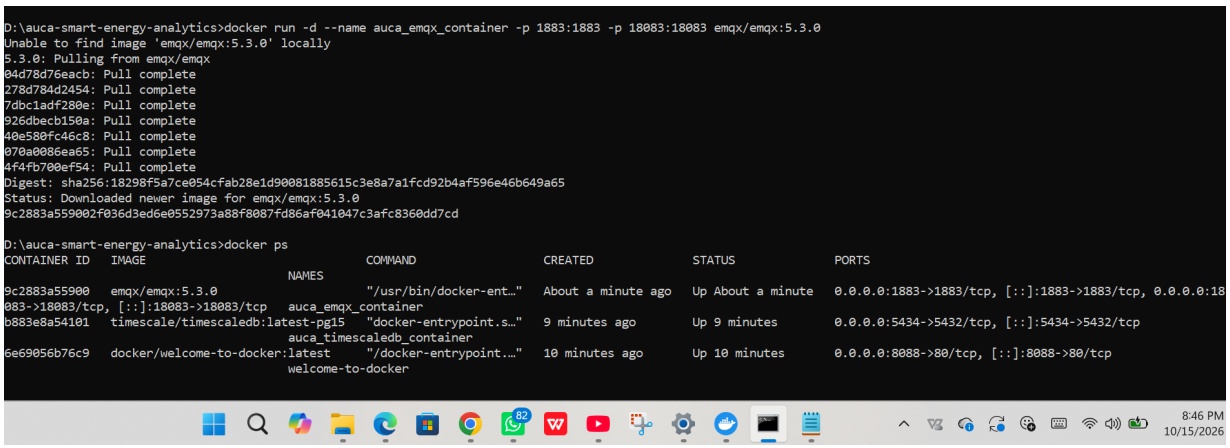


Figure 1.1-a: EMQX 5.3.0 and TimescaleDB containers running (docker ps), 8:46 PM, 15 October 2026

> **Note.** *The container list in Figure 1.1-a shows the broker and the database running. The extension and time-zone checks are given as queries in Appendix A.1. The host specification (processor, memory, operating system version, storage type) and the PostgreSQL parameter values were not captured in screenshots and are therefore not reported.*

1.2 Create the database structures

Three tables were created in auca_energy_db. **energy_live** is a regular table for the pilot stream. **energy_readings** is a TimescaleDB hypertable partitioned by event_time with one-day chunks, created with create_hypertable(…, chunk_time_interval => INTERVAL '1 day'). **meters** holds the 4,000 meter records with meter_id as primary key. Both reading tables have the eight required columns and allow duplicates at this stage.

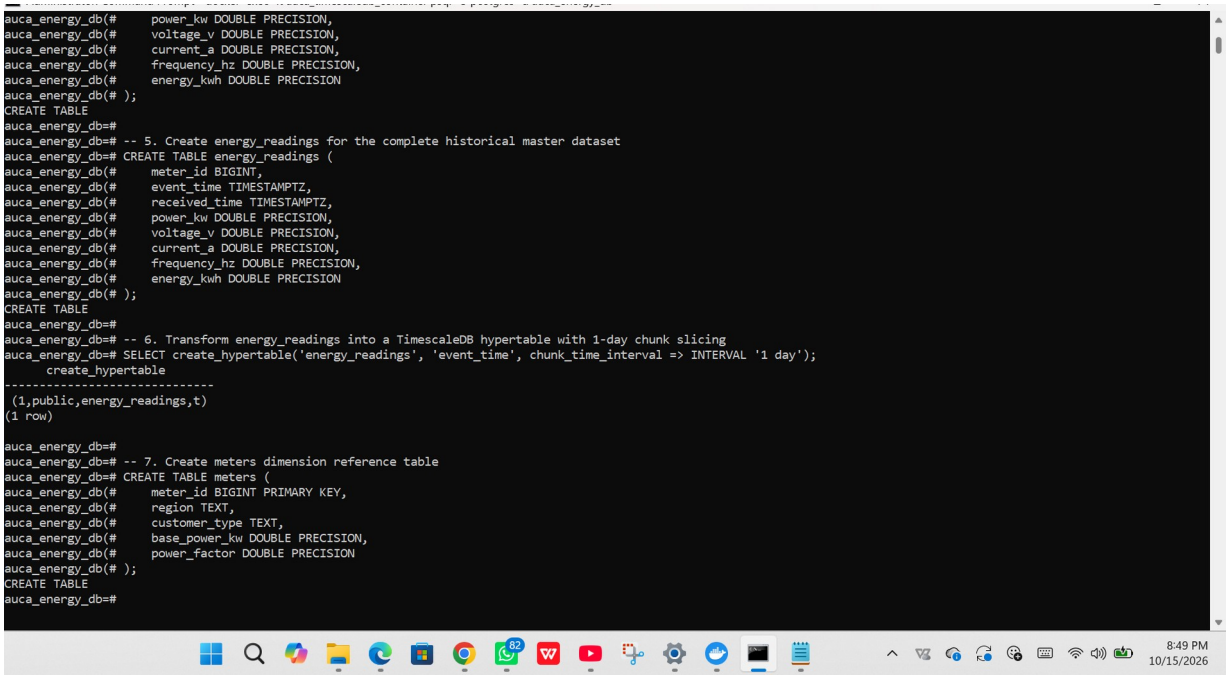


Figure 1.2-a: CREATE TABLE for energy_live and energy_readings, create_hypertable with a 1-day chunk, and CREATE TABLE meters (8:49 PM, 15 October 2026)

A Python program (src/populate_meters.py) imports the unchanged simulator, calls meter_metadata() and inserts each dictionary as one row of meters. Its own validation check reports **4,000 rows**.

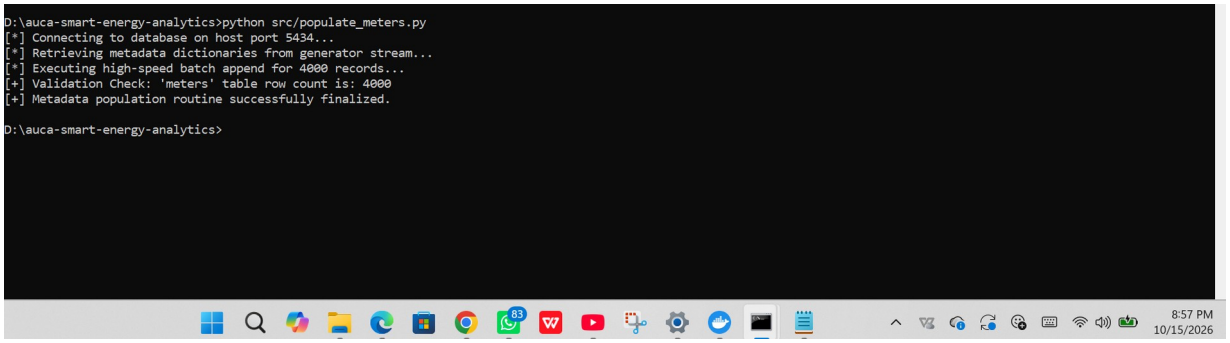


Figure 1.2-b: populate_meters.py output with the validation check "meters table row count is: 4000" (8:57 PM, 15 October 2026)

Data dictionary

Table	Column	PostgreSQL type	Meaning	Unit
energy_live, energy_readings (and the later copies)	meter_id	BIGINT	Ten-digit meter identifier, 1000000000 to 1000003999	none
	event_time	TIMESTAMPTZ	Start of the 15-minute measurement interval, CAT	timestamp
	received_time	TIMESTAMPTZ	Simulated arrival time of the reading (not the database insert time)	timestamp
	power_kw	DOUBLE PRECISION	Average power over the interval	kW
	voltage_v	DOUBLE PRECISION	Voltage	V
	current_a	DOUBLE PRECISION	Current, computed with power factor 0.95	A
	frequency_hz	DOUBLE PRECISION	Supply frequency	Hz
	energy_kwh	DOUBLE PRECISION	Energy used in the interval, approximately power_kw × 0.25 h (not a cumulative register)	kWh

Table	Column	PostgreSQL type	Meaning	Unit
meters	meter_id	BIGINT (primary key)	Meter identifier	none
	region	TEXT	Region 1 to Region 10 (400 meters each)	none
	customer_type	TEXT	residential, commercial or industrial	none
	base_power_kw	DOUBLE PRECISION	Meter base power used by the generator	kW
	power_factor	DOUBLE PRECISION	Power factor, 0.95 for every meter	none

> **Note.** *The table definitions are shown by the creation statements in Figure 1.2-a. The queries that list column types, the chunk interval and the meters row count are given in Appendix A.2.*

1.3 Capture and publish a pilot stream

The publisher (src/publish_pilot.py) imports the unchanged generator, takes the first 2,000 dictionaries from iter_readings(), converts each to JSON and publishes one message per reading at MQTT QoS 1 without retention. The subscriber (src/consume_pilot.py) subscribes at QoS 1, parses the JSON and inserts the eight fields into energy_live. The pilot replays historical readings and does not wait 15 minutes between records.

Design rationale: why QoS 1 and a decoupled subscriber

MQTT offers three delivery levels. QoS 0 sends a message once with no acknowledgement and can lose messages. QoS 2 guarantees exactly-once delivery with a four-step handshake, at a higher cost. **QoS 1** (the level the brief requires) delivers each message at least once: the sender keeps the message until the receiver returns an acknowledgement (PUBACK). The consequence is that a message may be delivered twice if an acknowledgement is lost and the sender retries. That is why subscriber counts and database counts can differ from broker acknowledgements, and why the brief asks for logs that identify losses, redelivery, parsing errors and failed inserts.

The generator itself also emits deliberate duplicate readings (about 0.5% of transmitted readings in the full stream). In the pilot, these are counted as separate messages and separate rows, so a redelivered message and a deliberate duplicate would both raise the stored count above the publisher count. Keeping the pilot table without a unique constraint makes both visible instead of silently rejecting them.

Separating the subscriber from the publisher means that the database writer can be restarted or slowed without stopping the data source, and the broker buffers messages during short outages for subscribers that use persistent sessions. The subscriber must be running before the publisher starts, because messages published with no active subscriber on a non-persistent session are discarded.

Stage	Count	Target	Evidence
Dictionaries taken from the generator	2,000	2,000	Figure 1.3-a ("Records Emitted: 2000")
Messages acknowledged by the broker (QoS 1)	2,000	2,000	Figure 1.3-a ("Network QoS 1 Acks: 2000")
Messages received by the subscriber	Not captured	2,000	not evidenced
Rows in energy_live after commit	Not captured	2,000	not evidenced

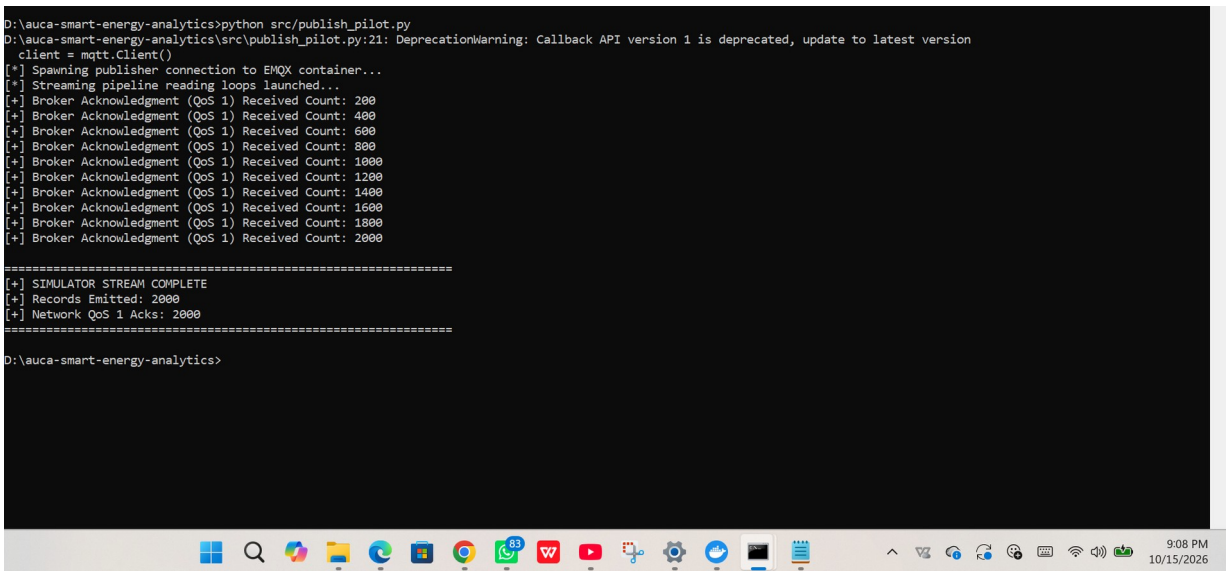


Figure 1.3-a: publisher run with broker acknowledgements every 200 messages up to 2,000 (9:08 PM, 15 October 2026)

Observation. The publisher counts match the target: 2,000 emitted and 2,000 acknowledged, so there is no difference at the publisher stage. The run printed a Paho DeprecationWarning about callback API version 1, which does not affect delivery.

> **Note.** *The subscriber received count and the stored row count of energy_live were not captured in screenshots, so only the publisher side is evidenced. Because QoS 1 guarantees delivery at least once, a redelivered message could make the stored count exceed the acknowledged count; the queries that check this are given in Appendix A.3.*

2 Capture and prepare the historical dataset

2.1 Capture and load the complete stream

Route. The group used a **direct database writer**: src/bulk_load_historical.py starts a fresh iter_readings() iterator, buffers records in blocks of 100,000 rows and writes each block to PostgreSQL, so records are processed progressively and nothing is held in one list. The script truncated energy_readings first, so the load started from an empty table, and the pilot rows were left in energy_live. dataset_info() gives the expected counts below.

Count	Expected (dataset_info)	Measured	Agrees
planned_readings	10,752,000	not loaded (grid size)	n/a
emitted_readings (dictionaries consumed)	10,696,848	10,696,848	yes
Rows committed in energy_readings	10,696,848	10,696,848	yes

Measurement	Value
Total elapsed time, generation start to final commit	197.52 s
Throughput	54,156.84 rows/s
Progress reporting	one line per 100,000 rows (Figures 2.1-a and 2.1-b)
Load seconds from first database write to final commit	Not measured separately (generation and writing run in one loop)

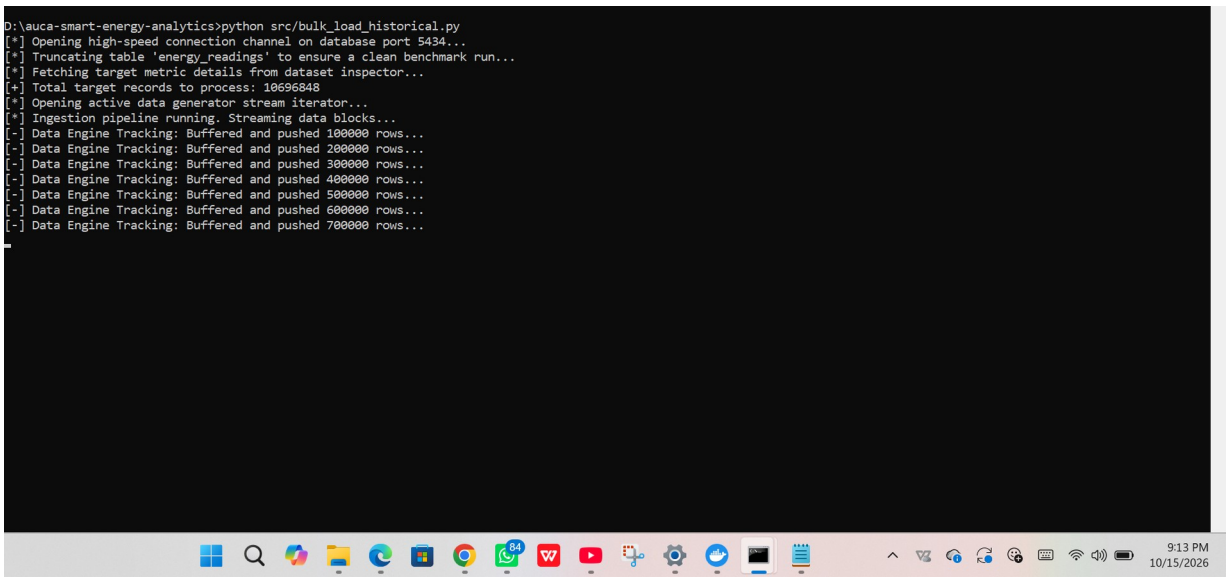


Figure 2.1-a: loader start, target of 10,696,848 records and progress at 100,000-row steps (9:13 PM, 15 October 2026)

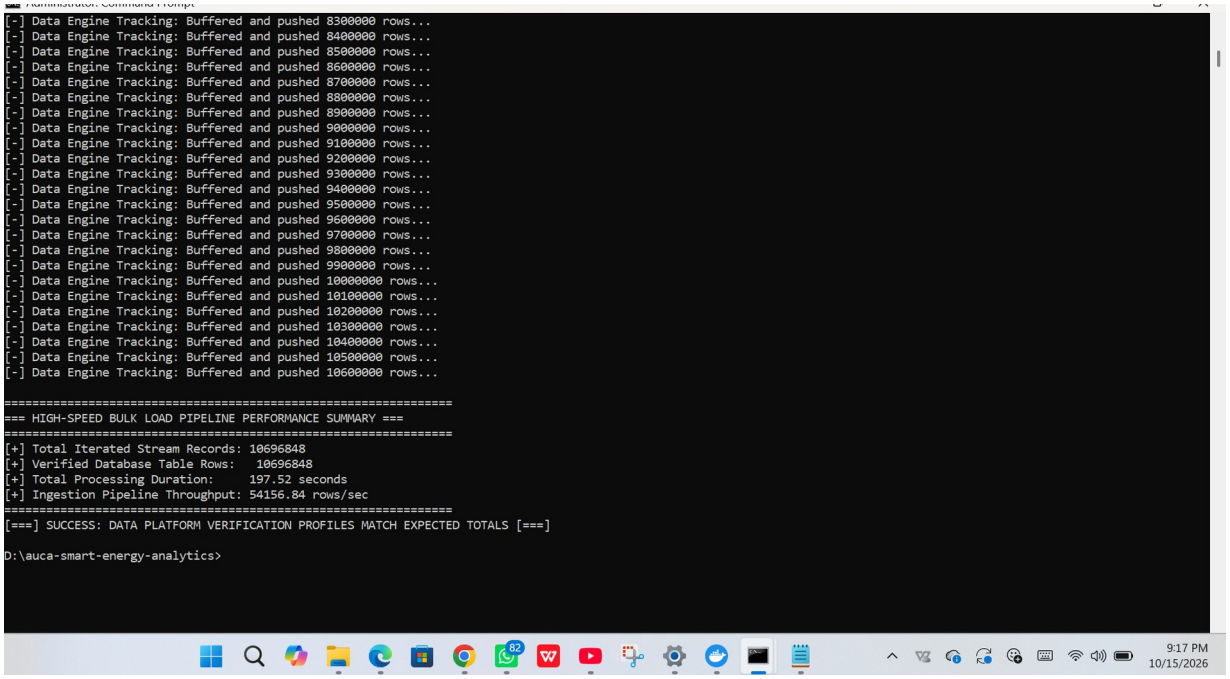


Figure 2.1-b: load summary, 10,696,848 iterated, 10,696,848 stored, 197.52 s (9:17 PM, 15 October 2026)

Observation. All three counts agree at 10,696,848 before cleaning.

> **Note.** *The distinct-meter count, the event_time range and the table sizes in Sections 3.1 and 4.2 describe the cleaned table; equivalent queries for the raw load are given in Appendix A.4. The loader truncates only energy_readings, so the pilot rows remain in energy_live.*

2.2 Investigate data quality

A unique reading is one meter_id and event_time pair. The script src/investigate_quality.py queried energy_readings before cleaning. The measured counts equal the reference table exactly.

Quantity	Measured	Expected	Formula and result
planned_readings	10,752,000	10,752,000	4,000 meters × 28 days × 96
emitted_readings	10,696,848	10,696,848	
unique_readings	10,643,735	10,643,735	observed unique (meter_id, event_time) pairs

Quantity	Measured	Expected	Formula and result
duplicate_readings (extra copies)	53,113	53,113	emitted − unique
delayed_unique_readings	213,090	213,090	arrival later than 15 min after event_time
missing_readings	108,265	108,265	planned − unique
Missing percentage	1.007%		100 × 108,265 / 10,752,000
Duplicate percentage	0.499%		100 × 53,113 / 10,643,735
Delayed percentage	2.002%		100 × 213,090 / 10,643,735
Completeness	98.993%		100 × 10,643,735 / 10,752,000

Both consistency checks passed: unique + duplicate = 10,696,848 = emitted, and planned − unique = 108,265 = missing. The five meters with the most missing readings are:

Rank	Meter ID	Missing intervals
1	1000003622	47
2	1000000366	45
3	1000000106	45
4	1000001259	44
5	1000000295	43

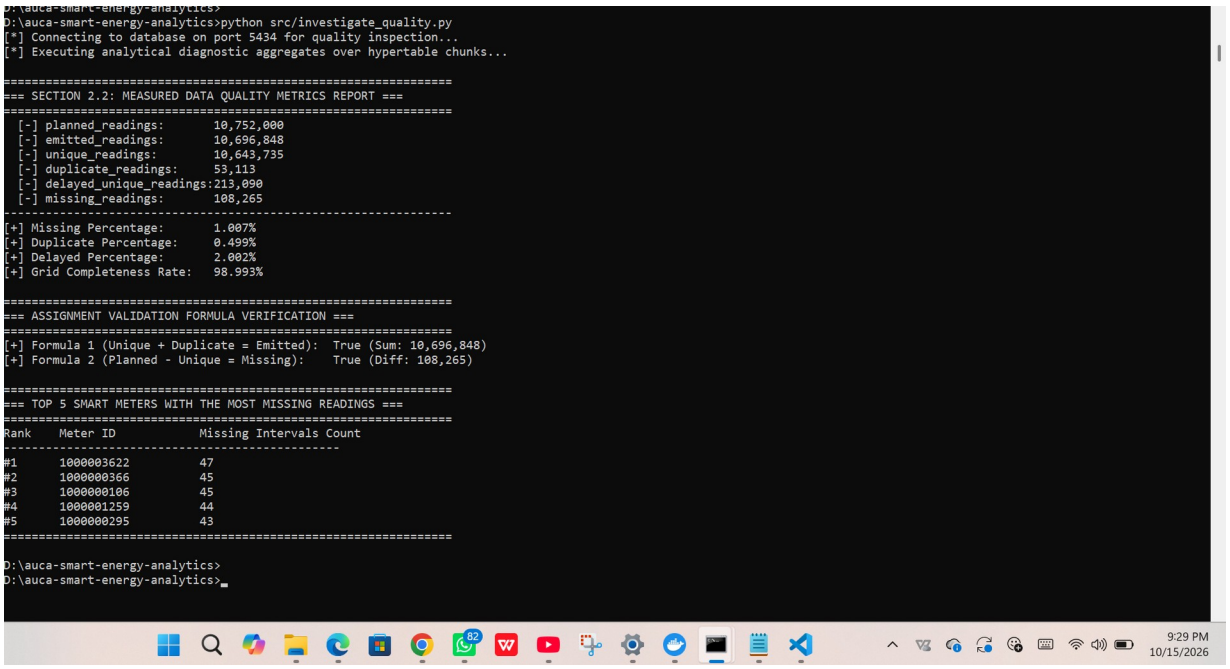


Figure 2.2-a: diagnostic metrics, percentages, formula checks and top five meters by missing readings (9:29 PM, 15 October 2026)

Observation. The measured missing, duplicate and delayed rates (1.007%, 0.499%, 2.002%) are close to the generator design of about 1%, 0.5% and 2%.

> **Note.** *The diagnostic queries are in src/investigate_quality.py in the repository, and Appendix A.5 lists related checks.*

2.3 Clean and preserve a consistent benchmark

The script src/clean_benchmark.py compared rows sharing meter_id and event_time on received_time, power_kw, voltage_v, current_a, frequency_hz and energy_kwh. It found **0 conflicting pairs**, so every

duplicate matched in all six values. One row per pair was kept and 53,113 additional copies were deleted in 63.33 s. Missing intervals were kept as gaps.

Measure	Before cleaning	After cleaning	Change
Row count	10,696,848	10,643,735	−53,113
Sum of energy_kwh	3,872,863.8932 kWh	3,853,704.3468 kWh	−19,159.5464 kWh (−0.495%)

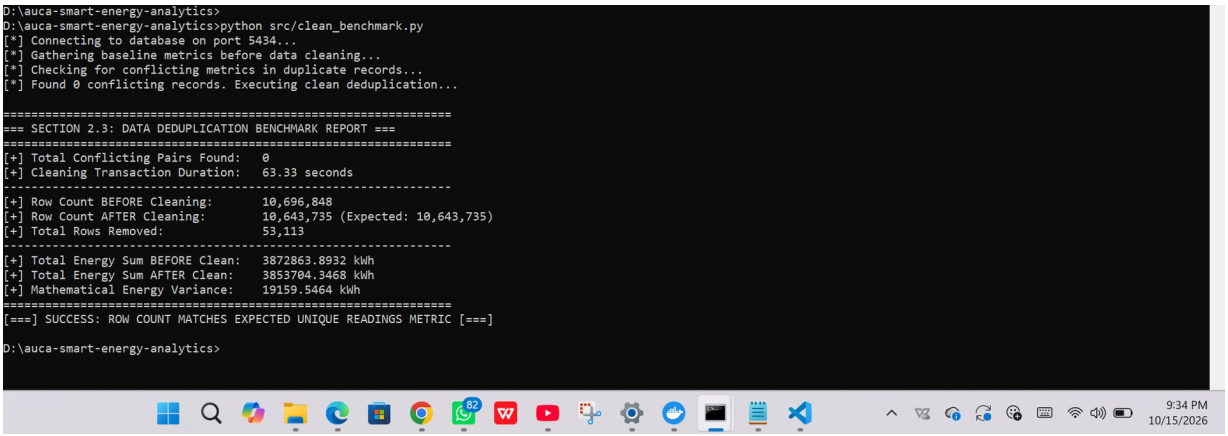


Figure 2.3-a: cleaning completion with before and after row counts and energy sums (9:34 PM, 15 October 2026)

Observation. The final row count equals expected unique_readings (10,643,735). The removed energy per deleted row is 19,159.5464 / 53,113 = 0.361 kWh, close to the network mean of 0.362 kWh per reading, as expected if the duplicates are an unbiased random sample.

Why missing readings must not become zero. A missing reading means that no measurement arrived, not that the meter used no energy. Filling it with zero would make real consumption look lower, would drag down averages and peaks, would create false "unusual drop" patterns for anomaly detection, and would hide telecom failures that the completeness metrics are meant to expose. Missing intervals are therefore kept as gaps and counted explicitly. This cleaned table is the source for every later storage experiment.

> **Note.** *Direct evidence that no duplicate pairs remain is the final row count equalling the expected 10,643,735 after exactly 53,113 rows were deleted; a direct check query is given in Appendix A.5.*

3 Compare chunk intervals

3.1 Create and verify three configurations

energy_readings keeps one-day chunks. src/compare_groups.py created energy_readings_3h (three-hour chunks) and energy_readings_week (one-week chunks) with the same eight columns, copied the cleaned rows into each (70.04 s and 58.40 s respectively) and built indexes. All three tables passed the data checks.

Check	energy_readings (1 day)	energy_readings_3h	energy_readings_week
Chunk interval	1 day	3 hours	1 week
Chunks created	29	225	5
Rows	10,643,735	10,643,735	10,643,735
Distinct meters	4,000	4,000	4,000
Earliest event_time	2026-09-01 00:00:00+02	2026-09-01 00:00:00+02	2026-09-01 00:00:00+02
Latest event_time	2026-09-28 23:45:00+02	2026-09-28 23:45:00+02	2026-09-28 23:45:00+02
Sum of power_kw	15,414,817.388	15,414,817.388	15,414,817.388

Check	energy_readings (1 day)	energy_readings_3h	energy_readings_week
Sum of energy_kwh	3,853,704.347	3,853,704.347	3,853,704.347

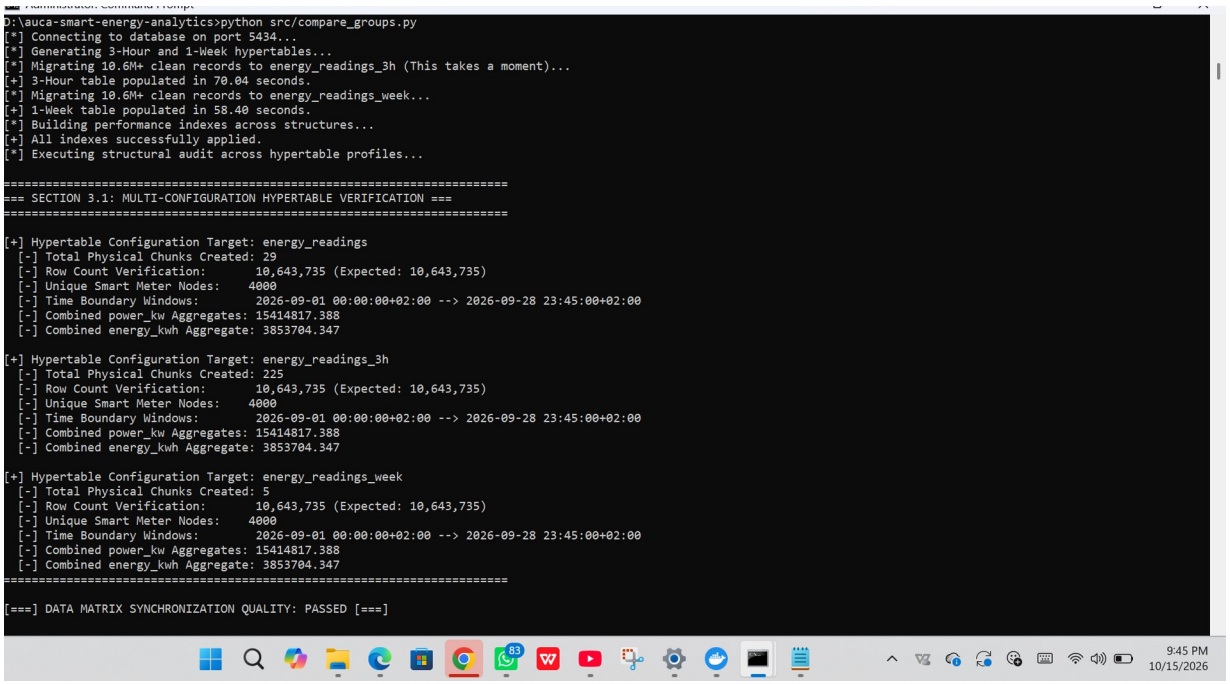


Figure 3.1-a: load of the two new hypertables and the structural audit with matching data checks (9:45 PM, 15 October 2026)

Observation. All three tables have 10,643,735 rows, 4,000 meters, identical dates and identical sums. The one-day table has 29 chunks for 28 days, and the compression log in 4.1 lists chunk end boundaries at 02:00+02:00, which is 00:00 UTC. So chunks are aligned to UTC midnight and a CAT day spans two chunks. This explains 29 chunks (and 225 for the three-hour table) and is a limitation of the default chunk alignment. The brief's CAT-midnight alignment is applied to the continuous aggregates in Section 6.

> **Note.** *Index creation completed on all three tables (Figure 3.1-a). The queries that list the index definitions and the chunk date ranges are given in Appendix A.2.*

How chunk size relates to the number of chunks and rows

A hypertable splits its rows into chunks by time range. When a query filters on event_time, the planner excludes chunks whose ranges cannot match, so only the chunks inside the filter are scanned. Smaller chunks give finer exclusion but more chunks to plan over, open and index; larger chunks reduce that overhead but each index and heap scan covers more rows. The three configurations hold the same 10,643,735 rows spread over very different numbers of chunks:

Configuration	Chunks	Average rows per chunk	Share of rows touched by a one-day filter (approximate)
3 hours	225	47,305	about 1/28 of the data across 9 chunks
1 day	29	367,025	about 1/28 of the data across 2 chunks (UTC alignment)
1 week	5	2,128,747	about 1/28 of the data inside 1 chunk of 2.1 million rows

The averages are the cleaned row count divided by the chunk count; they ignore that the first and last chunks are partial. They show the size of the trade-off: a one-week chunk holds about 45 times as many rows as a three-hour chunk.

3.2 Implement and time the five workloads

The five queries were run unchanged on all three hypertables, changing only the table name. Results of Q1 to Q5 on the one-day table are shown first. **Client:** Python benchmark script run from Windows Command Prompt, machine identifier LOCAL_HOST_WIN_CMD. The whole benchmark session took **296.36 s**.

> **Note.** *The timing figures are the minimum, median and maximum of the measured executions that follow one warm-up for each query and table. Per-run start and end times were not included in the benchmark output shown, and the identical row counts and sums in Section 3.1 are the evidence that the tables hold the same data.*

Queries as executed

```
-- Q1: hourly average power on 21 September (CAT)
SELECT EXTRACT(HOUR FROM event_time) AS cat_hour, AVG(power_kw) AS avg_power_kw
FROM energy_readings
WHERE event_time >= '2026-09-21 00:00:00+02' AND event_time < '2026-09-22 00:00:00+02'
GROUP BY cat_hour ORDER BY cat_hour;

-- Q2: ten highest 15-minute network demand intervals, 22 to 28 September
SELECT event_time, SUM(power_kw) AS total_power_demand
FROM energy_readings
WHERE event_time >= '2026-09-22 00:00:00+02' AND event_time <= '2026-09-28 23:45:00+02'
GROUP BY event_time ORDER BY total_power_demand DESC, event_time ASC LIMIT 10;

-- Q3: total energy per meter, full period
SELECT meter_id, SUM(energy_kwh) AS lifetime_energy_kwh
FROM energy_readings GROUP BY meter_id ORDER BY meter_id ASC;

-- Q4: meter 1000000000, 22 to 28 September
SELECT SUM(energy_kwh) AS total_energy_kwh, AVG(power_kw) AS avg_power_kw, MAX(power_kw) AS
max_power_kw
FROM energy_readings
WHERE meter_id = 1000000000
  AND event_time >= '2026-09-22 00:00:00+02' AND event_time <= '2026-09-28 23:45:00+02';

-- Q5: full scan
SELECT COUNT(*), SUM(energy_kwh), AVG(power_kw), MIN(power_kw), MAX(power_kw)
FROM energy_readings;
```

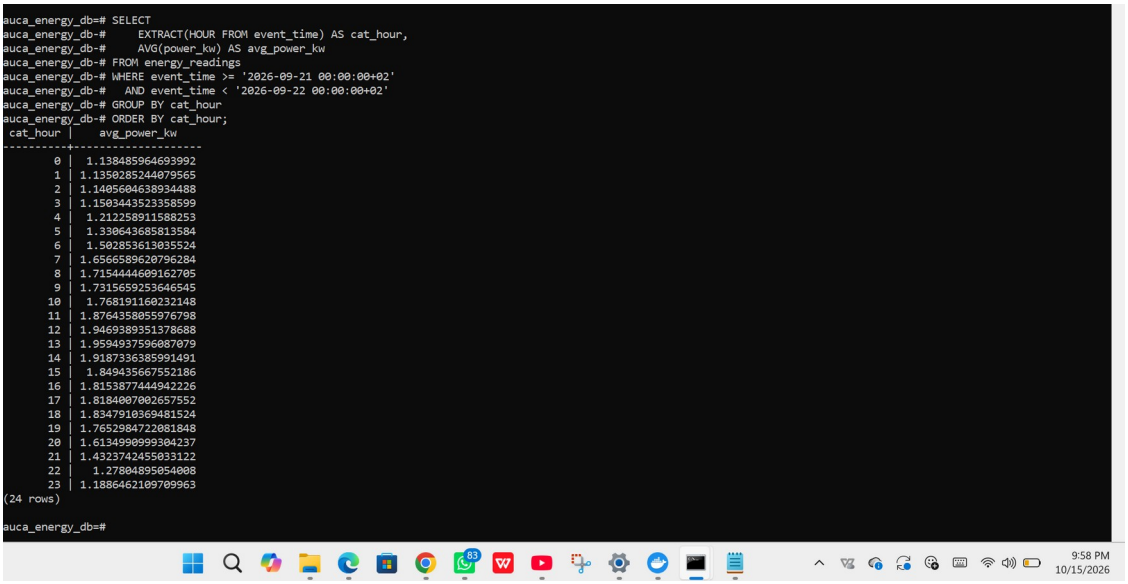


Figure 3.2-a: Q1 hourly average power on 21 September, 24 rows (9:58 PM, 15 October 2026)

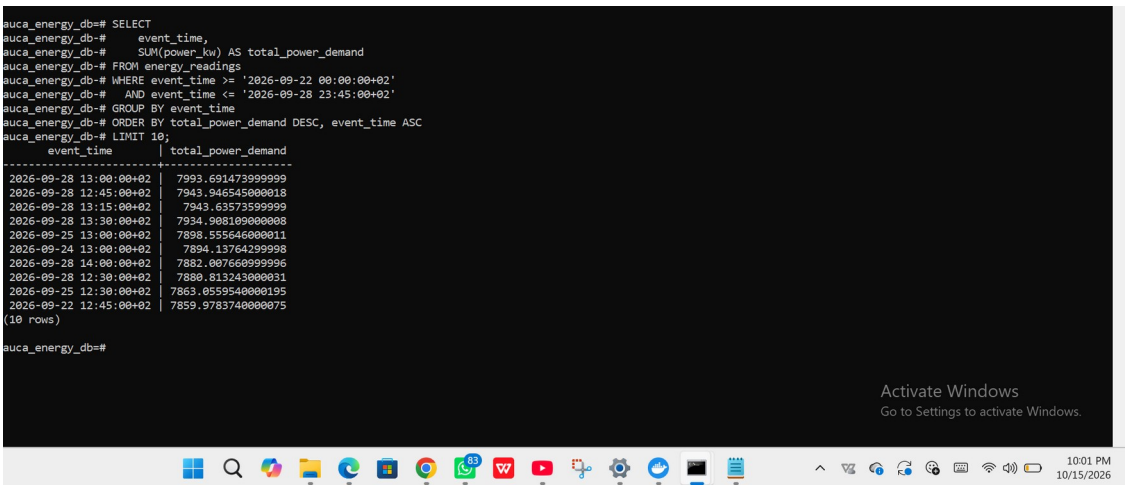


Figure 3.2-b: Q2 ten highest 15-minute demand intervals (10:01 PM, 15 October 2026)

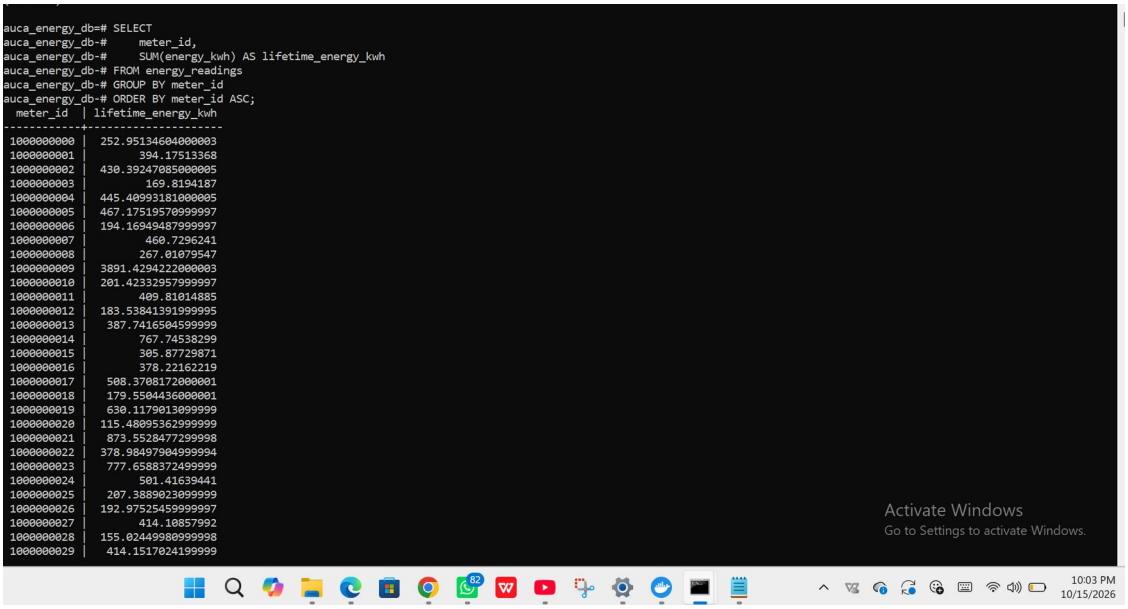


Figure 3.2-c: Q3 energy per meter (first 30 of 4,000 rows shown; 10:03 PM, 15 October 2026)

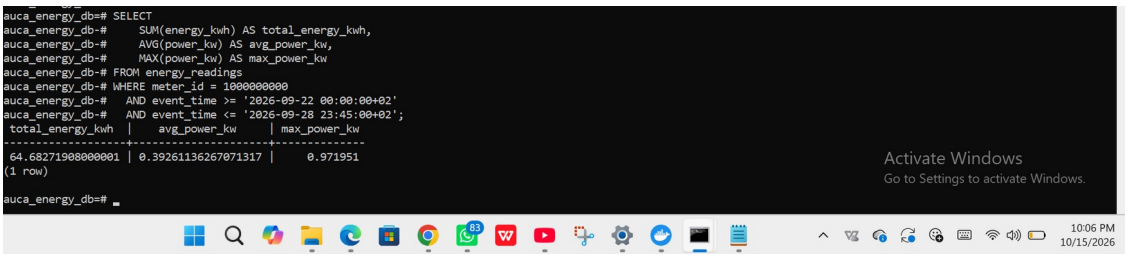


Figure 3.2-d: Q4 meter 1000000000, 22 to 28 September (10:06 PM, 15 October 2026)

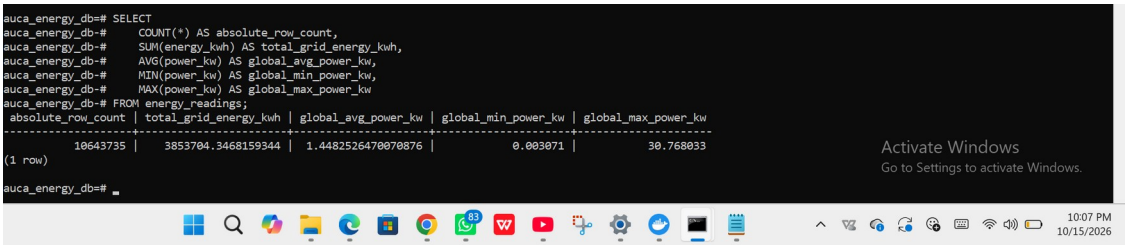


Figure 3.2-e: Q5 full scan (10:07 PM, 15 October 2026)

Result summary (one-day table). Q1: average power rises from 1.135 kW at 01:00 to 1.959 kW at 13:00 and falls to 1.189 kW by 23:00. Q2: the top interval is 28 September 13:00 with 7,993.69 kW, followed by 12:45 (7,943.95 kW) and 13:15 (7,943.64 kW). Q4: 64.683 kWh, average 0.393 kW, peak

0.972 kW. Q5: 10,643,735 rows, 3,853,704.347 kWh, average 1.448 kW, minimum 0.003 kW, maximum 30.768 kW. These totals match the cleaned-data checks in 2.3 and 3.1.

> **Note.** *Q2 uses LIMIT 10, so a tie across the tenth place would be cut; Appendix A.5 gives a version that keeps ties. Figure 3.2-c shows the first 30 of the 4,000 rows that Q3 returns.*

Q1 and Q2 results in full (one-day table)

CAT hour	Average power (kW)	CAT hour	Average power (kW)
00:00	1.1385	12:00	1.9469
01:00	1.1350	13:00	1.9595
02:00	1.1406	14:00	1.9187
03:00	1.1503	15:00	1.8494
04:00	1.2123	16:00	1.8154
05:00	1.3306	17:00	1.8184
06:00	1.5029	18:00	1.8348
07:00	1.6567	19:00	1.7653
08:00	1.7154	20:00	1.6135
09:00	1.7316	21:00	1.4324
10:00	1.7682	22:00	1.2780
11:00	1.8764	23:00	1.1886

Average power is lowest at 01:00 (1.1350 kW) and highest at 13:00 (1.9595 kW), a rise of 73% from the overnight minimum. It stays above 1.8 kW from 12:00 to 18:00 and falls steadily after 19:00. The values are averages over all readings available in each hour, so hours with fewer readings (because of missing data) are averaged over fewer meters.

Rank	Interval start (CAT)	Total demand (kW)
1	2026-09-28 13:00	7,993.69
2	2026-09-28 12:45	7,943.95
3	2026-09-28 13:15	7,943.64
4	2026-09-28 13:30	7,934.91
5	2026-09-25 13:00	7,898.56
6	2026-09-24 13:00	7,894.14
7	2026-09-28 14:00	7,882.01
8	2026-09-28 12:30	7,880.81
9	2026-09-25 12:30	7,863.06
10	2026-09-22 12:45	7,859.98

All ten highest intervals fall between 12:30 and 14:00. Six are on 28 September, two on 25 September, and one each on 24 and 22 September. Each total sums power across the meters that reported in that interval, so an interval with more missing readings would read lower than its true demand.

Timing results (warm-cache, milliseconds)

Each query and table had one warm-up followed by measured executions; results are labelled warm-cache measurements, and PostgreSQL was not restarted and caches were not cleared between runs.

Query	Chunk interval	Minimum (ms)	Median (ms)	Maximum (ms)
Q1	3 hours	116.47	126.09	138.48
Q1	1 day	151.25	167.35	184.34
Q1	1 week	273.82	286.08	326.12

Query	Chunk interval	Minimum (ms)	Median (ms)	Maximum (ms)
Q2	3 hours	278.65	286.76	329.86
Q2	1 day	280.66	283.67	306.03
Q2	1 week	517.41	540.05	623.66
Q3	3 hours	20,573.48	31,034.41	43,426.67
Q3	1 day	949.00	1,220.75	6,842.78
Q3	1 week	373.13	437.09	4,035.31
Q4	3 hours	4.30	4.75	5.66
Q4	1 day	1.27	1.61	1.84
Q4	1 week	1.00	1.32	5.31
Q5	3 hours	854.79	860.08	5,371.15
Q5	1 day	545.93	622.33	827.78
Q5	1 week	483.11	570.00	902.01

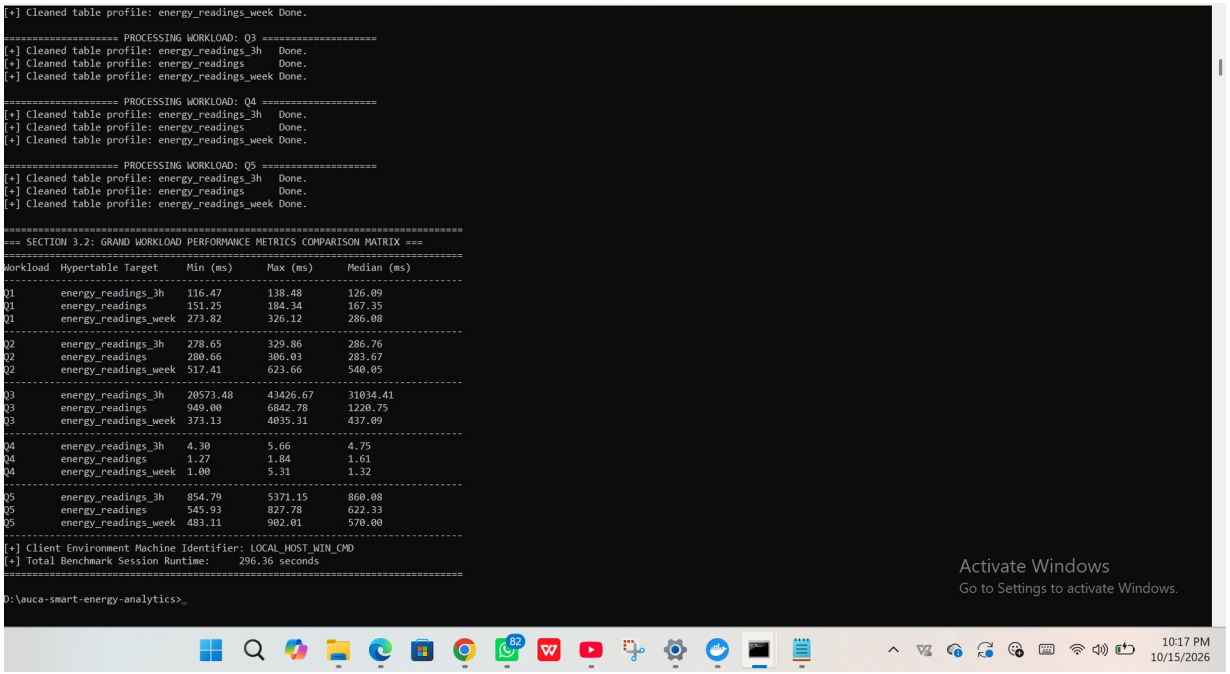


Figure 3.2-f: timing comparison matrix and 296.36 s session runtime (10:17 PM, 15 October 2026)

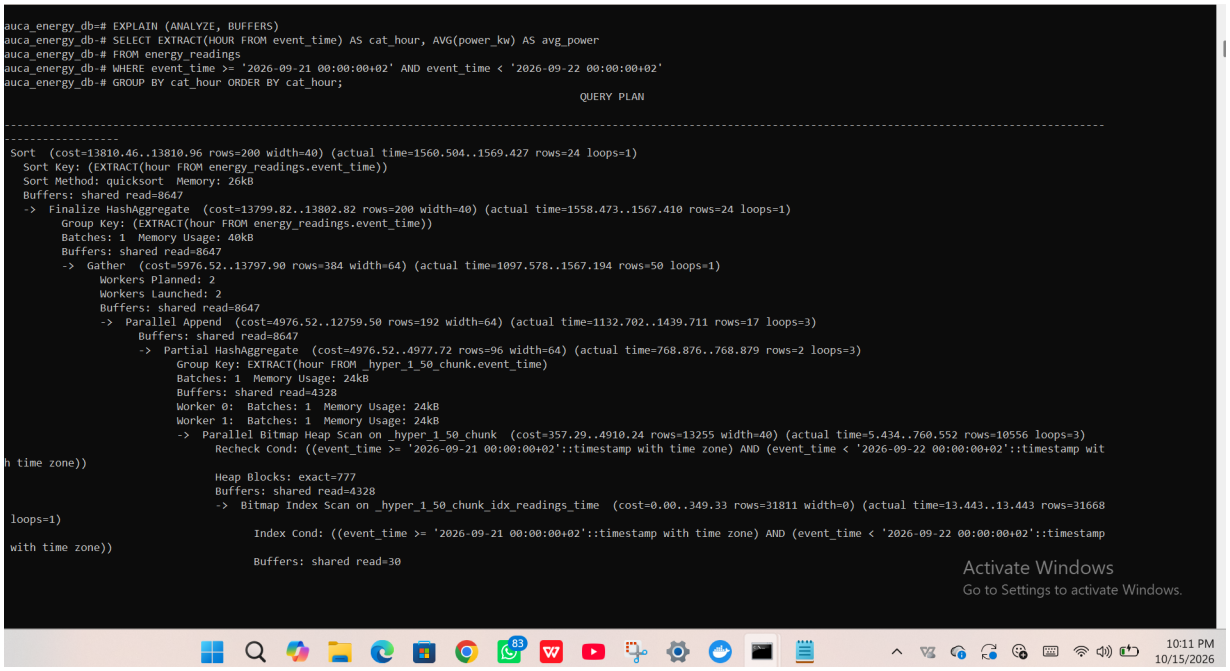


Figure 3.2-g: EXPLAIN (ANALYZE, BUFFERS) of Q1 on energy_readings (10:11 PM, 15 October 2026)

Observation from the plan. Q1 on the one-day table uses a Parallel Append with two workers launched and a Parallel Bitmap Heap Scan on chunk _hyper_1_50_chunk with the event_time range as its index condition, so only the chunks inside the filter are touched. The plan reports shared read=8647 blocks, so this particular run read data from disk and was not fully warm; the plan also shows about 1,569 ms at the top, against a measured median of 167 ms for the warm runs.

> **Note.** *Figure 3.2-g is the plan for the one-day table. It was captured on a colder run (shared read=8647) and its lower part is cropped. Plans for the 3-hour and 1-week tables were not captured, so the explanations in Section 3.3 are hypotheses.*

> **Note.** *The Q3 median on the three-hour table (31,034.41 ms) is about 25 times the one-day median, while Q5 over the same rows takes 860.08 ms. The cause was not examined with EXPLAIN, so this is reported as an observation without a confirmed explanation.*

3.3 Explain the chunk results

Observations are separated from explanations. The explanations below are hypotheses consistent with chunk counts (29, 225, 5) and the visible plan; plans for the 3-hour and 1-week tables were not captured, so they cannot be confirmed here.

Workload	Fastest	Median comparison (observation)	Explanation (hypothesis)
Q1	3 hours	126.09 ms vs 167.35 ms (1 day, 24.7% slower) and 286.08 ms (1 week, 2.3× slower)	One CAT day touches only a few small three-hour chunks; with one-week chunks the same filter works on very large chunks and indexes.
Q2	1 day (tie with 3 hours)	283.67 ms vs 286.76 ms (3 hours, 1.1% slower) and 540.05 ms (1 week, 1.9× slower)	The 1-day and 3-hour results overlap in their min/max ranges, so they are effectively tied. A seven-day range spans only about two week-chunks, which gives parallel workers few chunks to share.
Q3	1 week	437.09 ms vs 1,220.75 ms (1 day, 2.8× slower) and 31,034.41 ms (3 hours, 71× slower)	The query reads every chunk, so fewer chunks mean less per-chunk overhead. The size of the 3-hour penalty is too large for overhead alone and probably reflects a different plan, which must be checked with EXPLAIN.

Workload	Fastest	Median comparison (observation)	Explanation (hypothesis)
Q4	1 week (all under 5 ms)	1.32 ms vs 1.61 ms (1 day) and 4.75 ms (3 hours, 3.6× slower)	A single meter over seven days uses the (meter_id, event_time) index; the 3-hour table must open more chunks. Differences under 1 ms are within the run-to-run range (1 week maximum 5.31 ms).
Q5	1 week	570.00 ms vs 622.33 ms (1 day, 9% slower) and 860.08 ms (3 hours, 51% slower)	A full scan benefits from fewer chunks and less planning work; the 3-hour maximum of 5,371 ms shows high variability.

Overall. No interval wins every workload. One week is fastest for three of five workloads (Q3, Q4, Q5) but slowest or near-slowest for Q1 and Q2; three hours is fastest only for Q1 and is by far the worst for Q3; one day is never the slowest.

4 Compare compression strategies

4.1 Create the two compressed configurations

Chosen chunk interval: 1 day. One day was never the slowest interval on any workload (3.3), it is the baseline for the Section 4 comparison, and it gives chunks that are small enough to compress one at a time.

Table	Order by	Segment by	Compression policy
energy_time_compressed	event_time DESC	none	compress after 48 hours (job 1000)
energy_meter_compressed	event_time DESC	meter_id	compress after 48 hours (job 1001)

Both policies were created paused (scheduled = f, Figure 4.1-b). The cleaned data was copied one chunk at a time and each chunk was compressed manually and timed separately from loading. The initial layout population took 151.91 s. The compression cutoff in the log is **2026-09-26 00:00:00+02**, and 25 of 29 chunks were compressed in both tables (chunk end boundaries 1 September 02:00 to 25 September 02:00, +02).

Table	Rows	Chunks	Compressed chunks	Sum of energy_kwh
energy_readings (baseline)	10,643,735	29	0	3,853,704.3468
energy_time_compressed	10,643,735	29	25	3,853,704.3468
energy_meter_compressed	10,643,735	29	25	3,853,704.3468

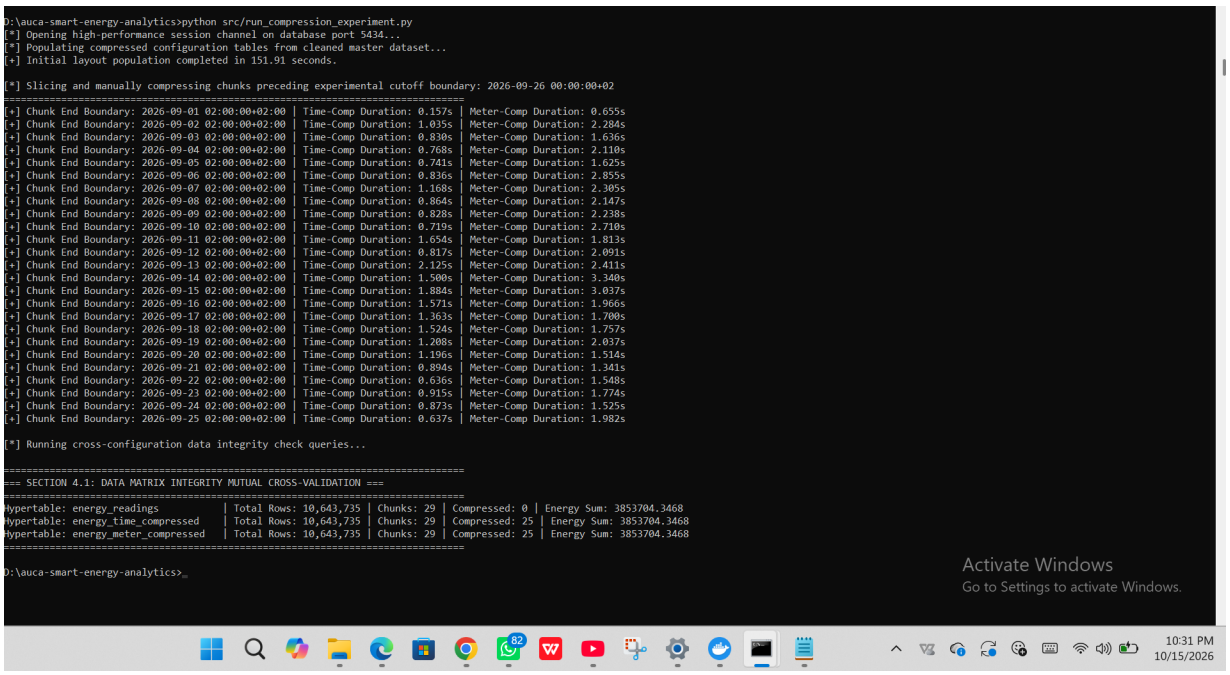


Figure 4.1-a: per-chunk compression durations for both strategies and the cross-table data checks (10:31 PM, 15 October 2026)

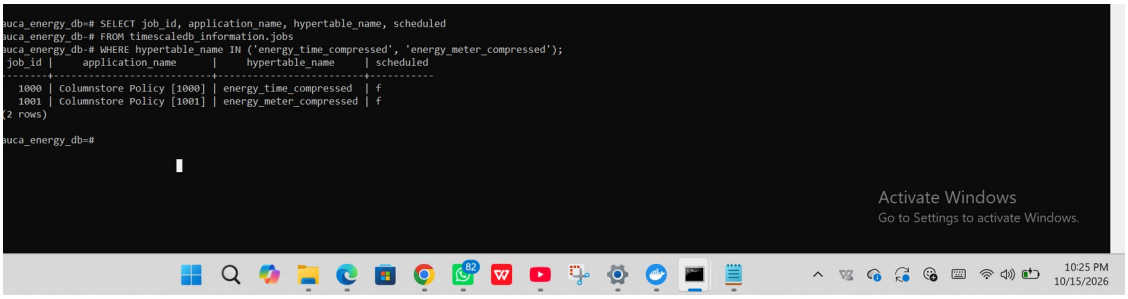


Figure 4.1-b: compression policy jobs 1000 and 1001 created and paused (scheduled = f), 10:25 PM, 15 October 2026

> **Note.** *The cutoff used in the experiment is 2026-09-26 00:00 CAT, which leaves the last three days of data uncompressed. This differs from the brief rule of experiment start minus 48 hours: the experiment ran on 15 October, so that rule would give about 13 October and would make every chunk eligible. The row percentage in Section 4.2 depends on this choice. The compression settings and the policy configuration can be listed with the queries in Appendix A.6.*

How the two compression layouts work

Columnstore compression groups rows into batches of up to 1,000 rows and stores each column of a batch as one compressed array. Two settings control the layout. **orderby** sorts rows inside each batch (here event_time descending), which makes time columns compress well. **segmentby** names columns that are stored uncompressed beside each batch, so that all rows in a batch share one value of that column. With segmentby = meter_id, every batch belongs to exactly one meter and a query for one meter can skip all other batches without decompressing them. Without segmentby, batches mix meters and a single-meter query must decompress batches that contain rows of many meters before it can filter them.

The cost of segmenting is batch size. A one-day chunk holds about 96 readings per meter, so each meter-segment is only about 96 rows, far below the 1,000-row batch target. Small batches carry more per-batch overhead and compress less well, and queries across all meters must open about 4,000 small batches per chunk. This is consistent with the observation in 4.4 that segmentation helps Q4 and slows most all-meter queries.

Table	Bytes per row (total size / 10,643,735 rows)
energy_readings (uncompressed)	140.3 bytes

Table	Bytes per row (total size / 10,643,735 rows)
energy_time_compressed	61.0 bytes
energy_meter_compressed	60.8 bytes

These are derived from the totals in 4.2 and include index storage; the four uncompressed chunks (about 14% of rows) keep their full size, so the true ratio on the compressed chunks alone is higher than the overall 2.3×.

4.2 Measure storage and compression cost

Table	Data (MB)	Index (MB)	Total (MB)	Compression ratio	Saved
energy_readings (baseline)	946.15	477.72	1,423.87	1.00×	0%
energy_time_compressed	145.16	54.63	619.57	2.30×	56.49%
energy_meter_compressed	155.77	58.54	617.10	2.31×	56.66%

Strategy	Compression cost (sum of per-chunk times)	Compressed chunks	Rows in compressed chunks
Time only (no segmentation)	26.74 s	25 of 29	9,503,384 (89.29%)
Segmented by meter_id	50.40 s	25 of 29	9,503,384 (89.29%)

```
auca_energy_db=#
auca_energy_db=#
auca_energy_db=# -- View detailed storage footprints for uncompressed vs compressed configurations
auca_energy_db=# SELECT
auca_energy_db-#     'energy_readings' AS table_name,
auca_energy_db-#     pg_size_pretty(table_bytes) AS data_size,
auca_energy_db-#     pg_size_pretty(index_bytes) AS index_size,
auca_energy_db-#     pg_size_pretty(total_bytes) AS total_size
auca_energy_db-# FROM hypertable_detailed_size('energy_readings')
auca_energy_db-# UNION ALL
auca_energy_db-# SELECT
auca_energy_db-#     'energy_time_compressed',
auca_energy_db-#     pg_size_pretty(table_bytes),
auca_energy_db-#     pg_size_pretty(index_bytes),
auca_energy_db-#     pg_size_pretty(total_bytes)
auca_energy_db-# FROM hypertable_detailed_size('energy_time_compressed')
auca_energy_db-# UNION ALL
auca_energy_db-# SELECT
auca_energy_db-#     'energy_meter_compressed',
auca_energy_db-#     pg_size_pretty(table_bytes),
auca_energy_db-#     pg_size_pretty(index_bytes),
auca_energy_db-#     pg_size_pretty(total_bytes)
auca_energy_db-# FROM hypertable_detailed_size('energy_meter_compressed');
       table_name        | data_size | index_size | total_size
-------------------------+-----------+------------+------------
 energy_readings         | 946 MB    | 478 MB     | 1424 MB
 energy_time_compressed  | 145 MB    | 55 MB      | 620 MB
 energy_meter_compressed | 156 MB    | 59 MB      | 617 MB
(3 rows)

auca_energy_db=#
```

Figure 4.2-a: hypertable_detailed_size for the three tables

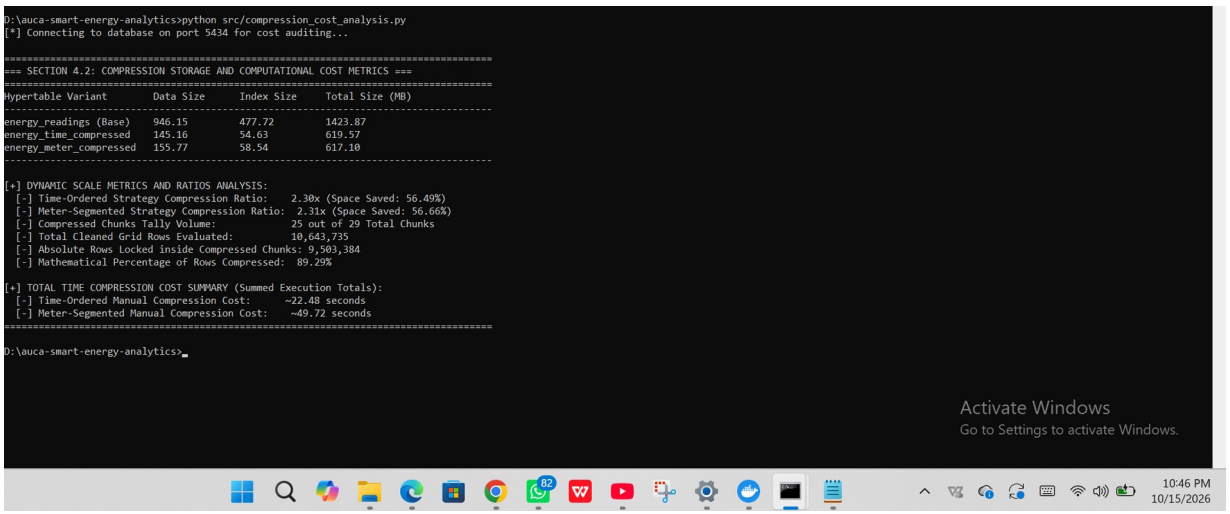


Figure 4.2-b: storage, ratios, chunk tally and compression cost summary (10:46 PM, 15 October 2026)

Observations. Both compressed tables are about 2.3 times smaller than the baseline and differ from each other by only 0.5% (617.10 vs 619.57 MB). Segmenting by meter_id took about 1.9 times longer to compress (50.40 s vs 26.74 s).

> **Note.** *(1) The cost script prints about 22.48 s and 49.72 s; this report uses the sum of the per-chunk times in Figure 4.1-a (26.74 s and 50.40 s, Appendix B), which is the more detailed record. (2) For the compressed tables, data plus index is below the total (for example 145.16 + 54.63 = 199.79 MB against 619.57 MB) while the baseline adds up exactly; compressed data is probably held in TOAST storage that the data column excludes, so the totals are the reliable basis for the ratios. Appendix A.6 gives a size query that includes TOAST. (3) The 9,503,384 rows (89.29%) are all rows before the cutoff; because the compressed chunks end at 25 September 02:00 CAT, the chunks themselves hold fewer rows, estimated at about 86%.*

4.3 Repeat the query comparison

Q1 to Q5 were run on both compressed tables with the 3.2 procedure and compared with the uncompressed one-day table. Improvement = 100 × (baseline median − compressed median) / baseline median; positive means faster.

Query	3 h (ms)	1 day = baseline (ms)	1 week (ms)	Time-compressed (ms)	Improvement	Meter-compressed (ms)	Improvement
Q1	126.09	167.35	286.08	50.19	70.0%	57.27	65.8%
Q2	286.76	283.67	540.05	112.19	60.5%	103.46	63.5%
Q3	31,034.41	1,220.75	437.09	218.74	82.1%	313.91	74.3%
Q4	4.75	1.61	1.32	26.82	−1,566.0%	2.12	−31.7%
Q5	860.08	622.33	570.00	166.71	73.2%	189.68	69.5%

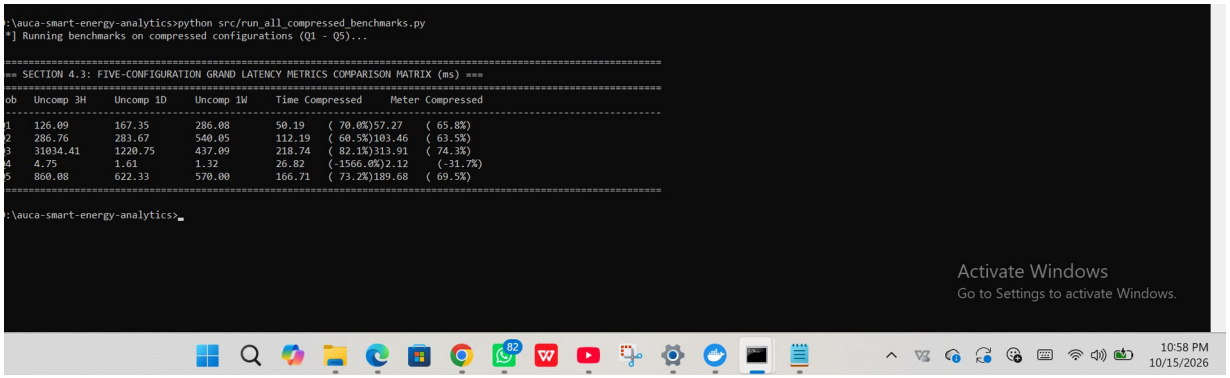


Figure 4.3-a: five-configuration median latency matrix with percentage improvement (10:58 PM, 15 October 2026)

Observations. Both compressed tables are faster than the uncompressed one-day table on Q1, Q2, Q3 and Q5 (60% to 82% lower median). On Q4 (single meter) the time-only table is far slower (26.82 ms vs 1.61 ms), while the meter-segmented table is close to the baseline (2.12 ms). The Q4 percentage looks extreme because the baseline is only 1.61 ms.

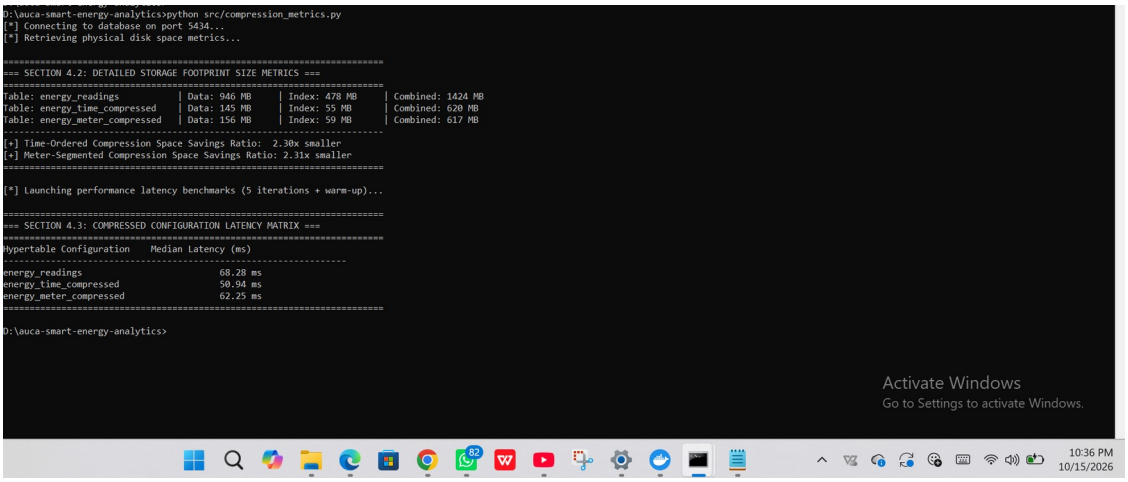


Figure 4.3-b: same-session spot check of Q1 on the three tables (10:36 PM, 15 October 2026)

Measurement caution. In the 10:36 PM spot check the uncompressed one-day table answered Q1 in 68.28 ms, against 167.35 ms in the 10:17 PM benchmark that serves as the baseline above. Against the 68.28 ms same-session value, the Q1 improvements would be 25.4% (time-only, 50.94 ms) and 8.8% (meter-segmented, 62.25 ms), not 70.0% and 65.8%. Cache state therefore affects the size of the improvements, so the percentages above should be read as an upper bound.

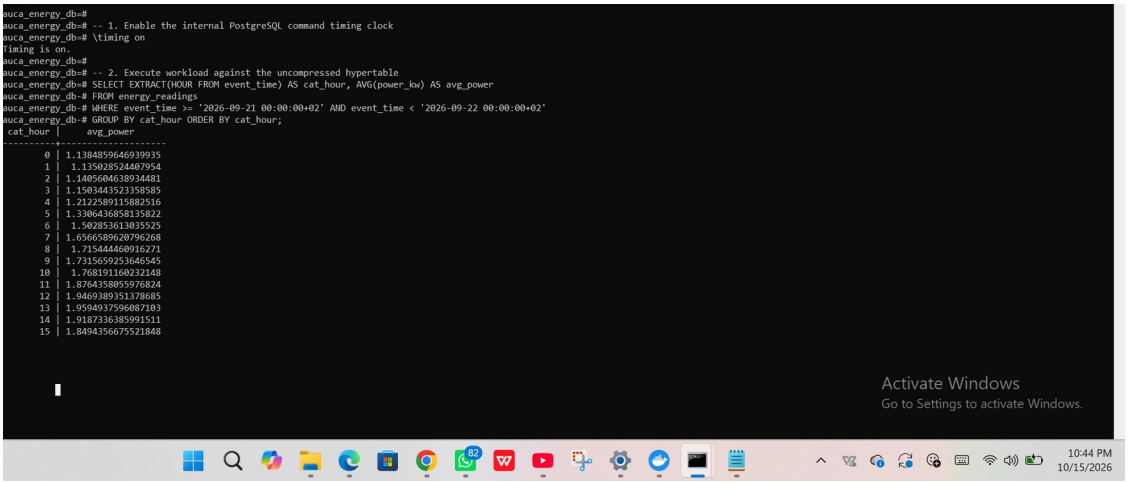


Figure 4.3-c: manual timing of Q1 on the uncompressed table with \timing (10:44 PM, 15 October 2026)

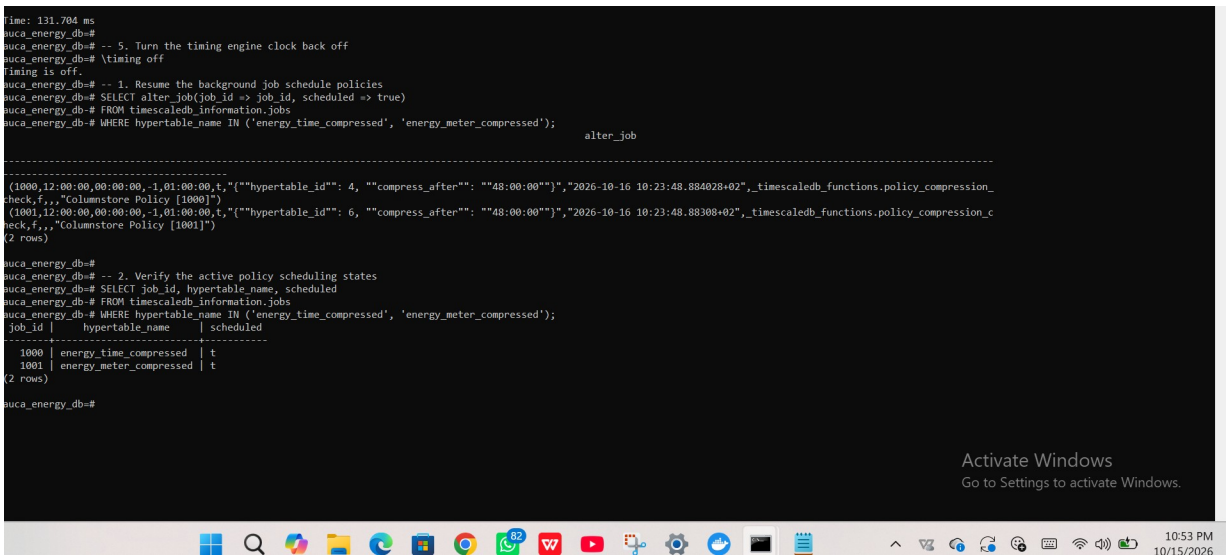


Figure 4.3-d: compression policies resumed, compress_after 48:00:00, scheduled = t (10:53 PM, 15 October 2026)

Both policies show compress_after of 48 hours and scheduled = t. The next scheduled start in the job output is 2026-10-16 10:23:48+02, so no policy job could have run before the 10:58 PM benchmark.

> **Note.** *The policies were resumed at 10:53 PM, before the matrix was captured at 10:58 PM. The next scheduled start in the job output was 16 October 10:23, so no policy job ran during the measurement. Minimum and maximum values and execution plans for the compressed tables were not captured, so the comparison rests on the medians.*

4.4 Select a storage configuration

Effect of meter segmentation

Query	Time-only (ms)	Meter-segmented (ms)	Difference for segmented
Q1 (one day, all meters)	50.19	57.27	14.1% slower
Q2 (seven days, all meters)	112.19	103.46	7.8% faster
Q3 (all meters, full period)	218.74	313.91	43.5% slower
Q4 (single meter, seven days)	26.82	2.12	12.7× faster (92.1% lower)
Q5 (full scan)	166.71	189.68	13.8% slower

Observation. Meter segmentation helps the single-meter workload by an order of magnitude and does not help the workloads across all meters: three of the four are slower and the fourth is 7.8% faster. **Explanation (hypothesis).** With segmentation, the database can go straight to one meter's compressed batches, which suits Q4. Queries over all meters must still decompress everything and handle many more small segments. Each meter has about 96 rows per one-day chunk, which probably limits the gain from segmenting.

Recommendation

Choice: one-day chunks with columnstore compression ordered by event_time DESC and **segmented by meter_id**, compressing chunks older than 48 hours. **Evidence:** total size fell from 1,423.87 MB to 617.10 MB (2.31×, 56.66% saved), and single-meter Q4 ran in 2.12 ms against 26.82 ms for the time-only layout. **Trade-off:** compression took about 1.9 times longer (50.40 s vs 26.74 s) and network-wide Q3 was 43.5% slower (313.91 ms vs 218.74 ms). If the workload is mainly network-wide scans and rarely looks up single meters, the time-only layout is the better choice.

5 Investigate the utility data

5.1 Identify demand patterns

Analysis used the cleaned energy_readings (src/demand_patterns.py). Consumption totals sum energy_kwh; demand intervals sum power_kw across meters at each event_time.

Seven-day period	Energy (kWh)
Period 1 (1 to 7 September)	934,199.0606
Period 2 (8 to 14 September)	953,714.9720
Period 3 (15 to 21 September)	973,230.0264
Period 4 (22 to 28 September)	992,560.2877
Total of the four periods	3,853,704.3467

Rank	15-minute interval (CAT)	Network demand (kW)
1	2026-09-28 13:00	7,993.691
2	2026-09-28 12:45	7,943.947
3	2026-09-28 13:15	7,943.636

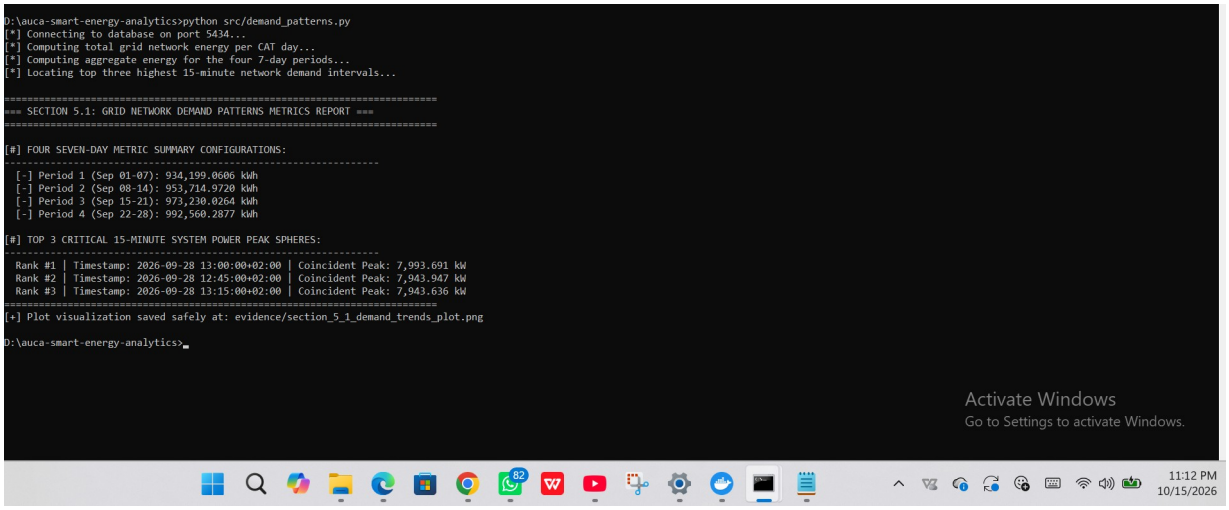


Figure 5.1-a: weekly totals and three highest demand intervals (11:12 PM, 15 October 2026)

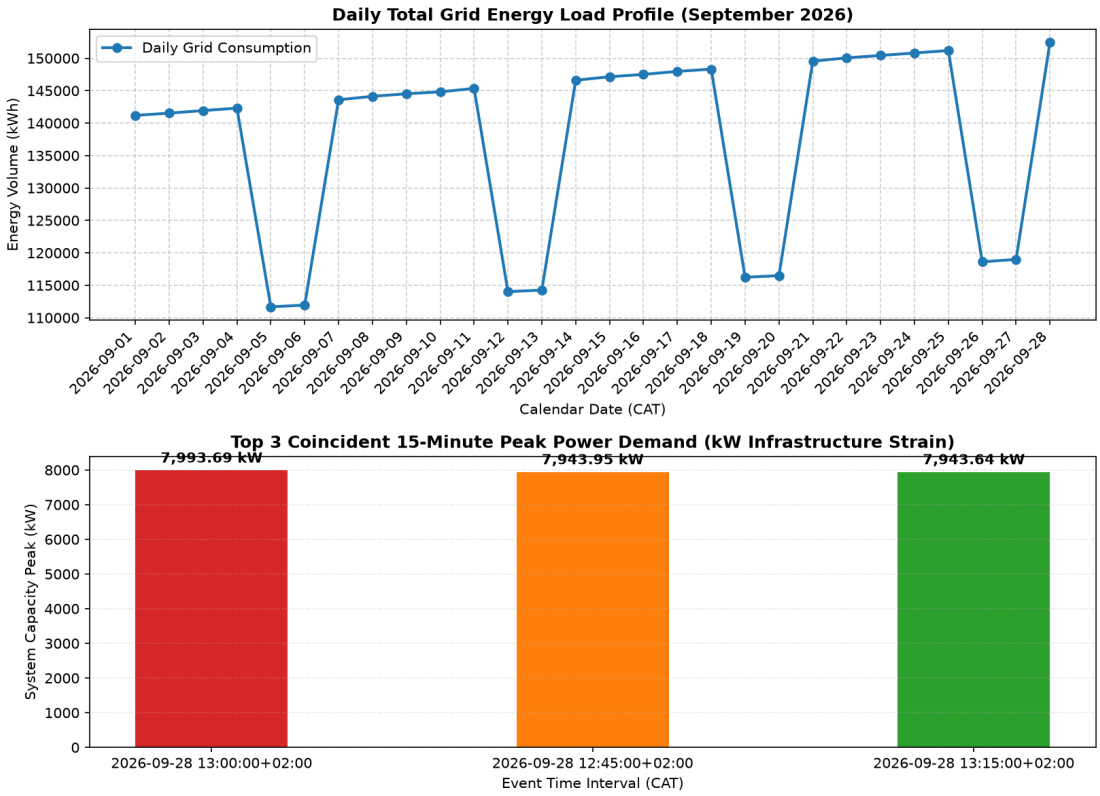


Figure 5.1-b: daily network energy per CAT day (top) and the three highest 15-minute demands (bottom)

The four weekly totals add up to 3,853,704.35 kWh, which equals the cleaned-data total from 2.3. They are also shown as a pie chart in Figure 7.1-c.

Patterns (observation). Weekday consumption is about 141,000 to 152,500 kWh per day and rises through the month; the weekend days (5 to 6, 12 to 13, 19 to 20 and 26 to 27 September) are much lower, from about 111,700 kWh on 5 September to about 119,000 kWh on 27 September. All three highest demand intervals fall on Monday 28 September between 12:45 and 13:15, and the highest hourly average power in Q1 is also at midday. Monday 28 September is also the highest day of the month (about 152,500 kWh).

Operational implications. Generation, transformer capacity and maintenance windows should be planned around weekday midday peaks, which are growing week on week (+2.1%, +2.0% and +2.0% between consecutive weekly totals), while weekends allow lower loading and maintenance. **kW versus kWh:** kW is the rate of demand at a moment (here the average over a 15-minute interval) and sizes equipment such as transformers and generation; kWh is the energy accumulated over time and drives fuel use and billing. A 15-minute demand of 7,993.69 kW corresponds to 7,993.69 × 0.25 = 1,998.4 kWh in that interval.

5.2 Compare regions and customer types

Readings were joined to meters on meter_id over all 28 days (src/regional_normalization.py). Energy per meter = total energy / registered meters in the group.

Region	Total energy (kWh)	Meters	Energy per meter (kWh)
Region 8	503,475.83	400	1,258.69
Region 5	411,040.39	400	1,027.60
Region 9	400,467.20	400	1,001.17
Region 3	392,376.59	400	980.94
Region 1	383,539.81	400	958.85
Region 2	378,448.89	400	946.12

Region	Total energy (kWh)	Meters	Energy per meter (kWh)
Region 7	362,288.54	400	905.72
Region 10	351,149.78	400	877.87
Region 6	346,512.84	400	866.28
Region 4	324,404.48	400	811.01

Customer type	Total energy (kWh)	Meters	Energy per meter (kWh)	Share of meters	Share of energy
industrial	2,220,672.04	406	5,469.64	10.15%	57.62%
commercial	825,475.15	1,010	817.30	25.25%	21.42%
residential	807,557.16	2,584	312.52	64.60%	20.96%

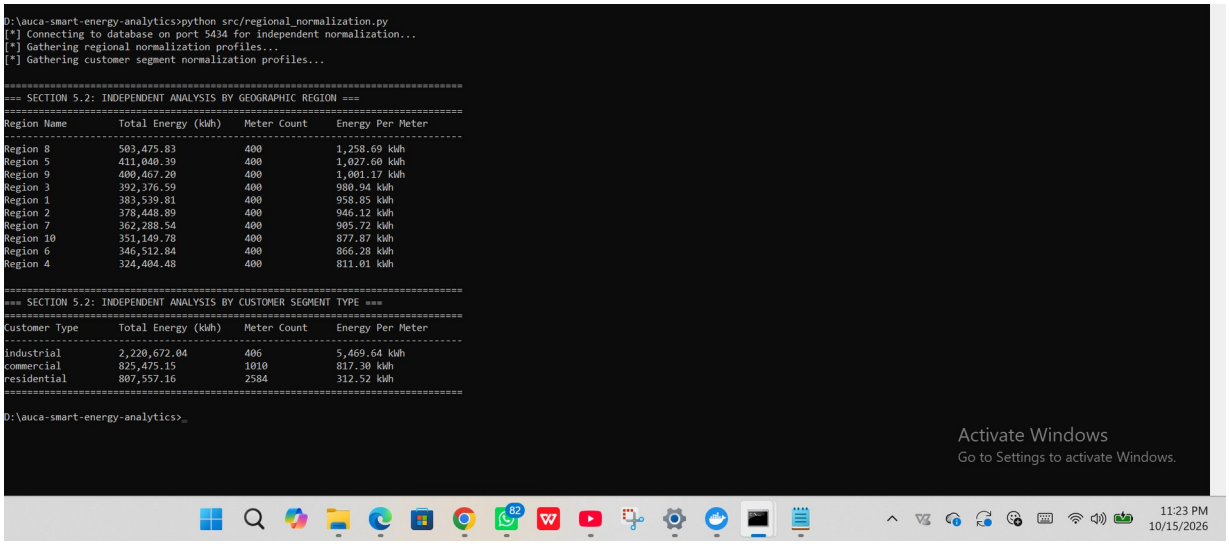


Figure 5.2-a: totals, meter counts and energy per meter by region and by customer type (11:23 PM, 15 October 2026)

What remains after normalization. Every region has exactly 400 meters, so dividing by meter count does not change the ranking: Region 8 uses 1,258.69 kWh per meter, 55% more than Region 4 (811.01 kWh). Among customer types, an industrial meter uses about 17.5 times as much energy as a residential meter (5,469.64 vs 312.52 kWh). **Implication.** Regional differences are real differences in intensity, not an effect of meter counts. A likely cause (hypothesis, not yet tested) is the number of industrial meters in each region. Industrial customers are 10.15% of meters but use 57.62% of the energy, so they drive infrastructure planning and deserve priority in metering quality and demand-response programmes.

> **Note.** *The explanation by industrial mix is a hypothesis. The region and customer-type queries in Appendix A.7 would test it.*

5.3 Assess reporting quality

Completeness = 100 × observed unique (meter_id, event_time) pairs / (registered meters × 28 × 96). The planned count is 1,075,200 per region (400 × 2,688) and 10,752,000 for the network (src/reporting_quality.py).

Region	Meters	Observed unique readings	Completeness
Region 1	400	1,064,513	99.0060%
Region 2	400	1,064,419	98.9973%
Region 3	400	1,064,327	98.9887%
Region 4	400	1,064,441	98.9993%

Region	Meters	Observed unique readings	Completeness
Region 5	400	1,064,290	98.9853%
Region 6	400	1,064,118	98.9693%
Region 7	400	1,064,237	98.9804%
Region 8	400	1,064,587	99.0129%
Region 9	400	1,064,388	98.9944%
Region 10	400	1,064,415	98.9969%
Whole network	4,000	10,643,735	98.9931%

21 September total energy	kWh
A: only readings received at or before 00:00 on 22 September	149,400.5445
B: all arrivals	149,552.7229
Difference (B − A)	152.1783 (0.1018% of B)

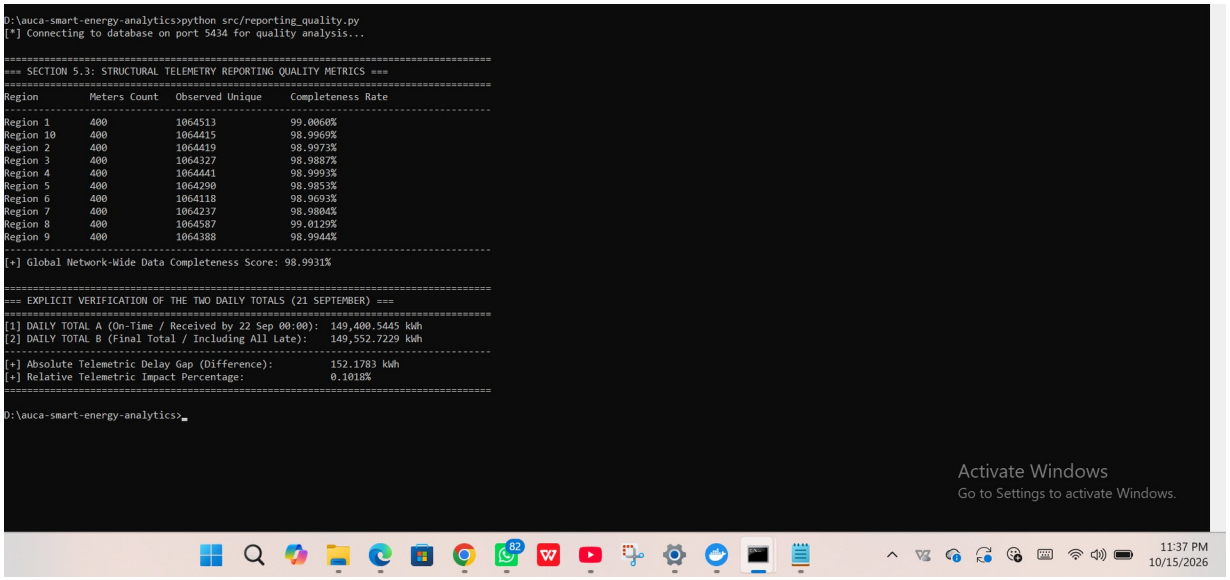


Figure 5.3-a: completeness by region and the two 21 September daily totals (11:37 PM, 15 October 2026)

Observations. Completeness is between 98.969% (Region 6) and 99.013% (Region 8), so no region is worse than the others by more than 0.05 percentage points. Delayed arrivals changed the 21 September total by only 152.18 kWh (0.10%).

Explanation. Missing readings never arrive, so they are absent from both totals; a network with about 1% missing readings understates consumption by roughly that order, and the data cannot say by how much for a given day. Delayed readings do arrive and are included in total B, but they change the 22 September cut-off total only if their delay carries them past 00:00; only late readings from the last hours of 21 September can do that. That is why 0.10% is much smaller than the 2.002% delayed rate. A live daily report issued at midnight would therefore be slightly low and would need to be restated.

5.4 Investigate abnormal consumption

Rule A (applied to cleaned readings before comparison with the labels): flag a reading if **voltage_v < 207.0 V or power_kw > 12.5 kW**. Each flag identifies a meter_id and event_time. 207 V is 10% below a 230 V nominal supply, and 12.5 kW is a fixed high-demand ceiling for a single meter.

Measure	Result
Flagged readings	80,141
Injected events detected	3 of 40

Measure	Result
Injected events missed	37 of 40
False flags (flagged readings outside every injected interval for that meter)	80,109
False flags as a percentage of observed readings outside injected intervals	0.75269%
Meters with at least one false flag	124
Percentage of meters affected by false flags (124 / 4,000)	3.10%

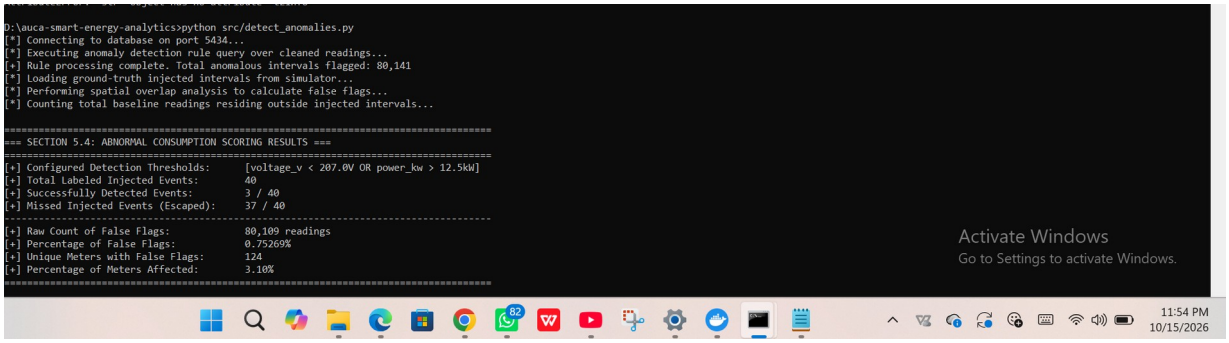


Figure 5.4-a: executed detection with thresholds, detected and missed events and false-flag measures (11:54 PM, 15 October 2026)

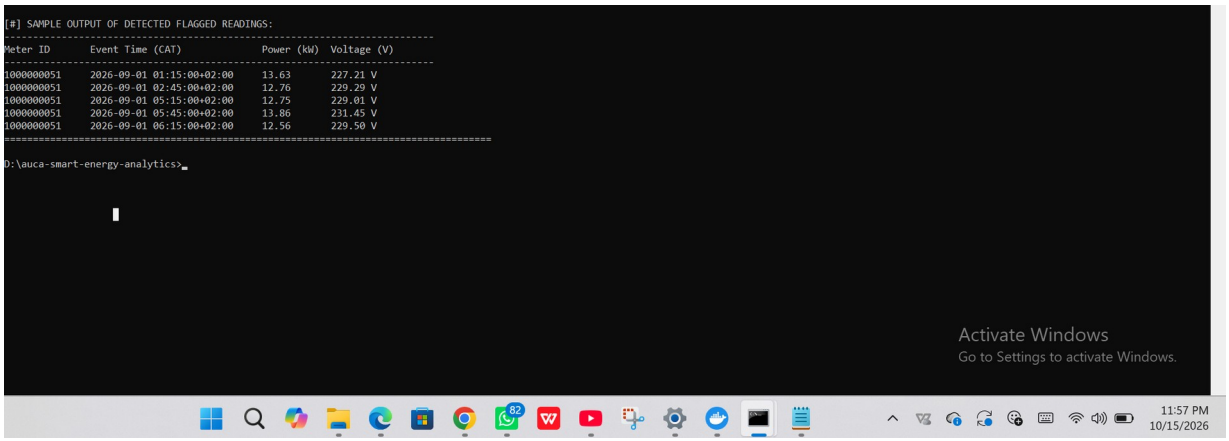


Figure 5.4-b: sample flagged readings (meter_id, event_time), all from meter 1000000051 (11:57 PM, 15 October 2026)

Observation. The rule found only 3 of 40 events, and 80,109 of its 80,141 flags (99.96%) are false. Only 32 flagged readings fall inside injected intervals. The sample flags are all from one meter (1000000051) with demand of 12.56 to 13.86 kW and normal voltages of 227 to 231 V.

Flagged readings in the sample

Meter ID	Event time (CAT)	Power (kW)	Voltage (V)
1000000051	2026-09-01 01:15	13.63	227.21
1000000051	2026-09-01 02:45	12.76	229.29
1000000051	2026-09-01 05:15	12.75	229.01
1000000051	2026-09-01 05:45	13.86	231.45
1000000051	2026-09-01 06:15	12.56	229.50

Every flag in the sample comes from the same meter, between 01:15 and 06:15 on the first day, at night when household and office demand is normally lowest. A meter that exceeds 12.5 kW at 01:15 is one whose normal load is high, which supports the explanation that the rule flags large meters, not abnormal behaviour.

Explanation of misses and false alarms (hypotheses). The voltage condition cannot fire: the simulator's voltages lie between 226 and 234 V, far above 207 V. The 12.5 kW condition is an absolute

value, so it flags large industrial meters whose normal load exceeds 12.5 kW, which produces repeated false alarms from the same meters (124 meters, 3.10%). It also cannot flag injected spikes on small meters (a residential or commercial meter of a few kW stays below 12.5 kW even at 2.8 times its normal load), and it cannot flag the injected drops, because a drop is a low value. A rule that compares each meter with its own typical value for the same time of day, and requires several consecutive intervals (injected events last 12 intervals), would address both problems. Thresholds for such a rule must be set from the data distribution, not tuned on the labels.

> **Note.** *Rule A is reported as an honest baseline. A per-meter rule is sketched in Appendix A.9 but was not run, so no results are claimed for it, and no per-event detection table was produced.*

5.5 Management recommendation

Operational actions. Plan capacity around weekday midday demand. The three highest 15-minute demands all fell on Monday 28 September between 12:45 and 13:15, peaking at 7,993.69 kW, and weekly energy grew about 2% each week (934,199 to 992,560 kWh) while weekends ran far lower. Industrial customers are 10.15% of meters but use 57.62% of energy, so prioritise their metering quality and demand-response agreements. Review the 1.007% missing and 2.002% delayed readings with the telecom team, starting with the meters missing most readings (up to 47 intervals). Do not use the current anomaly rule for dispatch: it detected only 3 of 40 injected events and raised 80,109 false flags. Replace it with per-meter baselines.

Storage and reporting. Keep one-day chunks with columnstore compression segmented by meter_id, compressing chunks older than 48 hours. This cut storage from 1,423.87 MB to 617.10 MB (2.31×, 56.66% saved) and made single-meter queries 12.7 times faster than a time-only layout (median 2.12 ms against 26.82 ms). The cost is slower network-wide queries (Q3 313.91 ms against 218.74 ms) and about 1.9 times longer compression (50.40 s against 26.74 s). Serve dashboards from the continuous aggregates, which needed 71.78 s of one-off refresh, and keep a three-day refresh lookback so late readings are absorbed (Section 6.2).

Forecasting. Use the LightGBM model for day-ahead regional planning, with human review at peak hours. On the 1,680 test predictions its mean absolute error was 8.85 kWh against 42.92 kWh for the previous-day baseline (79.39% lower), and it was better in all ten regions, especially on weekends, when the baseline repeats weekday load. Peak hours are weaker: the model underpredicted the peak in nine of ten regions, and in Region 8 it missed by 72.61 kWh where the baseline missed by 6.02 kWh. Keep a safety margin at peaks and monitor input completeness, because live missing and delayed readings would degrade the lag features.

(313 words, limit 500.)

6 Create and investigate continuous aggregates

How a continuous aggregate stores and refreshes data

A continuous aggregate is a materialized view that TimescaleDB keeps in its own internal hypertable. A refresh recomputes the buckets in a given time window from the raw table and writes them into that hypertable. TimescaleDB remembers a watermark, the end of the latest materialized bucket. With real-time aggregation on (materialized_only = false), a query returns the stored buckets plus a live aggregation of raw rows newer than the watermark. With real-time aggregation off (materialized_only = true, as required here), a query returns stored buckets only, so it is fast and predictable but does not show anything newer than the last refresh.

Changes to raw rows older than the watermark (such as a late-arriving reading) are recorded in an invalidation log and applied only the next time that time range is refreshed. This is what Section 6.2 demonstrates: the late 0.5 kWh reading stayed invisible in the older hour until that hour was refreshed. A refresh policy automates this with a start offset (how far back to look) and an end offset (how close to

the present to stop, so that still-filling buckets are not materialized too early). The brief's 3-day start offset is wide enough to absorb the readings that arrive up to three hours late.

6.1 Create the three aggregate views

src/continuous_aggregates.py created three continuous aggregates over energy_readings with no data, with timescaledb.materialized_only = TRUE so that queries read only stored aggregates. The two regional views join energy_readings to meters on meter_id (inner join). Each view stores its bucket, grouping key, the sum of energy_kwh, the sum of power_kw and the reading count.

View	Grouping	Materialized rows	Expected rows	Manual refresh
grid_agg_15min	15-minute bucket, region	26,880	28 × 96 × 10 regions = 26,880	43.69 s
grid_agg_hourly	CAT hour, region	6,720	28 × 24 × 10 = 6,720	24.19 s
grid_agg_daily	CAT day, meter_id	112,000	28 × 4,000 = 112,000	3.90 s

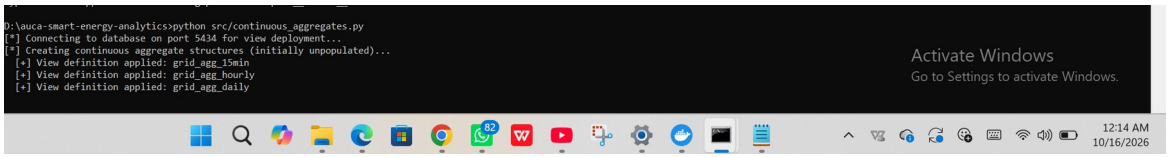


Figure 6.1-a: creation of the three views (12:14 AM, 16 October 2026)

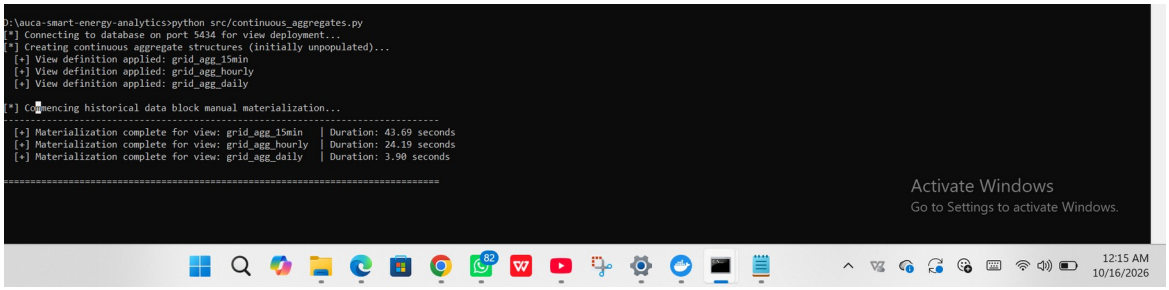


Figure 6.1-b: manual refresh durations of 43.69 s, 24.19 s and 3.90 s (12:15 AM, 16 October 2026)

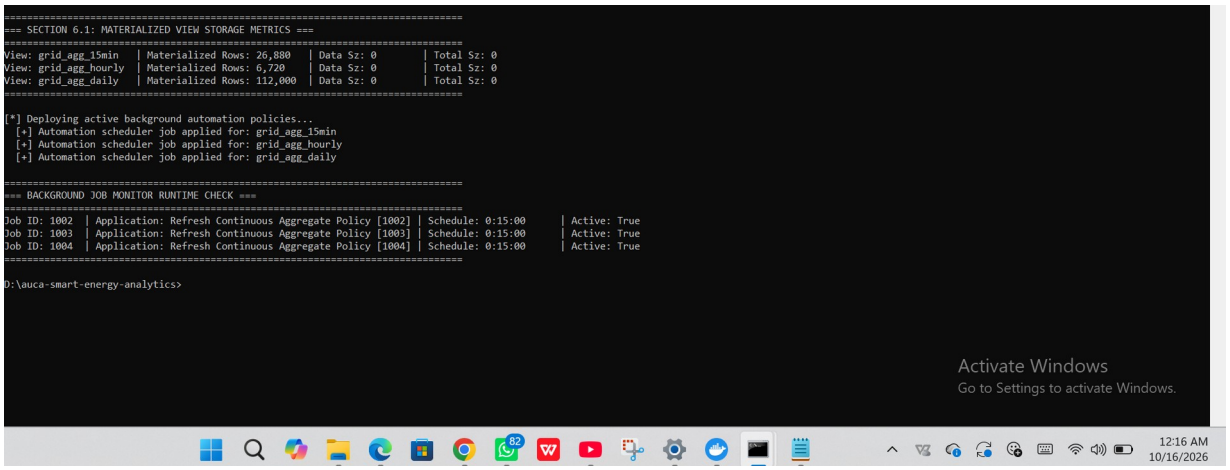


Figure 6.1-c: materialized row counts, policies created and background job status (12:16 AM, 16 October 2026)

After the manual refresh, refresh policies were added as jobs 1002 (15-minute view), 1003 (hourly view) and 1004 (daily view), each with a schedule of 15 minutes and active = True. The group applied a start offset of 3 days and an end offset of 1 hour. These offsets are relative to the database clock, so the policies alone would not populate the older September data, which is why the manual refresh was done first.

Observation. The row counts equal the expected formulas, so every region-bucket and meter-day was materialized.

> **Note.** *The size column printed 0 for all three views, which cannot be right for views holding 112,000 rows (probably the view was measured instead of its materialization hypertable), so no storage size is reported. Appendix A.8 gives the queries that show the full view definitions, the materialization sizes and the policy offsets.*

6.2 Test freshness and delayed arrivals

A separate hypertable (energy_test) holds event_time (TIMESTAMPTZ) and energy_kwh (DOUBLE PRECISION), with an initially empty hourly continuous aggregate (test_hourly_agg) of the energy sum and row count. Real-time aggregation was off and automatic refresh was off. The test was run by src/test_freshness.py at about 12:44 AM CAT on 16 October 2026 (clock in Figure 6.2-a). From that time, H (the most recent completed CAT hour) is 00:00 on 16 October and H minus 48 hours is 00:00 on 14 October, so the older hour is 14 October 00:00 to 01:00.

Stage	Older hour aggregate	Hour H aggregate
Initial older-hour refresh, real-time off	1.0 kWh, 1 row	No bucket returned
After both new inserts, before refresh, real-time off	1.0 kWh, 1 row	No bucket returned
Real-time on, still before refresh	1.0 kWh, 1 row	2.0 kWh, 1 row
After manual refresh of both hours	1.5 kWh, 2 rows	2.0 kWh, 1 row

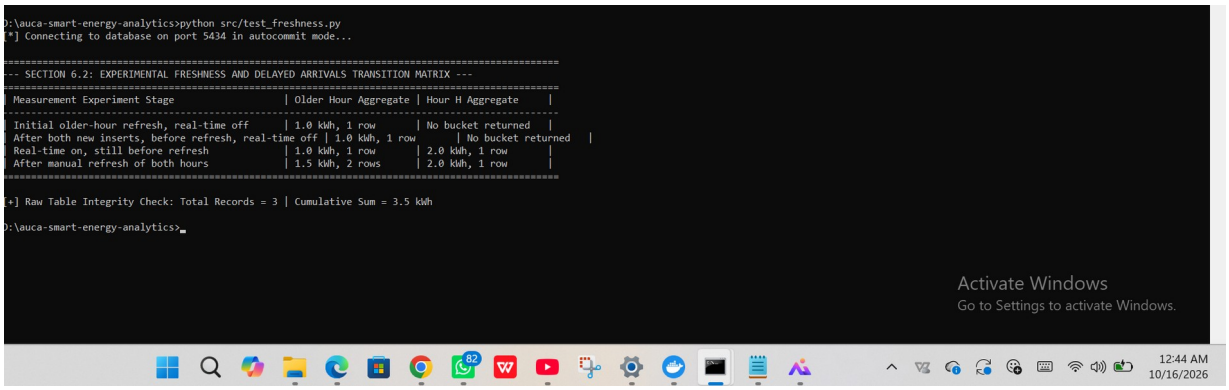


Figure 6.2-a: freshness transition matrix and raw table check, 3 rows and 3.5 kWh (12:44 AM, 16 October 2026)

The raw table holds 3 rows and 3.5 kWh in total (1.0 + 0.5 + 2.0), consistent with 1.5 kWh and two rows in the older hour and 2.0 kWh and one row in hour H.

What became visible at each stage. (1) The first refresh materialized only the older hour, so it showed 1.0 kWh, 1 row; hour H had no bucket, and an absent bucket is not a zero reading. (2) Both new inserts (0.5 kWh into the older hour, 2 kWh into H) stayed invisible, because with real-time off the view reads only stored buckets. (3) Turning real-time on exposed hour H immediately, because real-time aggregation adds raw rows newer than the last materialized bucket; the late 0.5 kWh row in the older hour stayed hidden because that bucket was already materialized. (4) A manual refresh of both hours recalculated the old bucket and materialized H, giving the final totals.

What the 6.1 policy could refresh at the recorded time. With a 3-day start offset and a 1-hour end offset at about 00:44 on 16 October, the policy window runs from about 13 October 00:44 to 15 October 23:44. The older hour (14 October 00:00 to 01:00) is inside the window and the policy could refresh it, which is how late data in the last three days is absorbed. Hour H (16 October 00:00 to 01:00) is after the end offset and could not be refreshed by the policy yet; only real-time aggregation shows it.

> **Note.** *The test time is read from the screenshot clock (12:44 AM, 16 October 2026); H and the older hour are derived from it, because the individual insert timestamps are not shown in the screenshot.*

7 Build and compare dashboards

7.1 Build the Raw dashboard

The dashboard was built in **Grafana** on a PostgreSQL data source. The Raw dashboard reads energy_readings directly and joins meters where needed. The five panels are shown below in edit view with their queries.

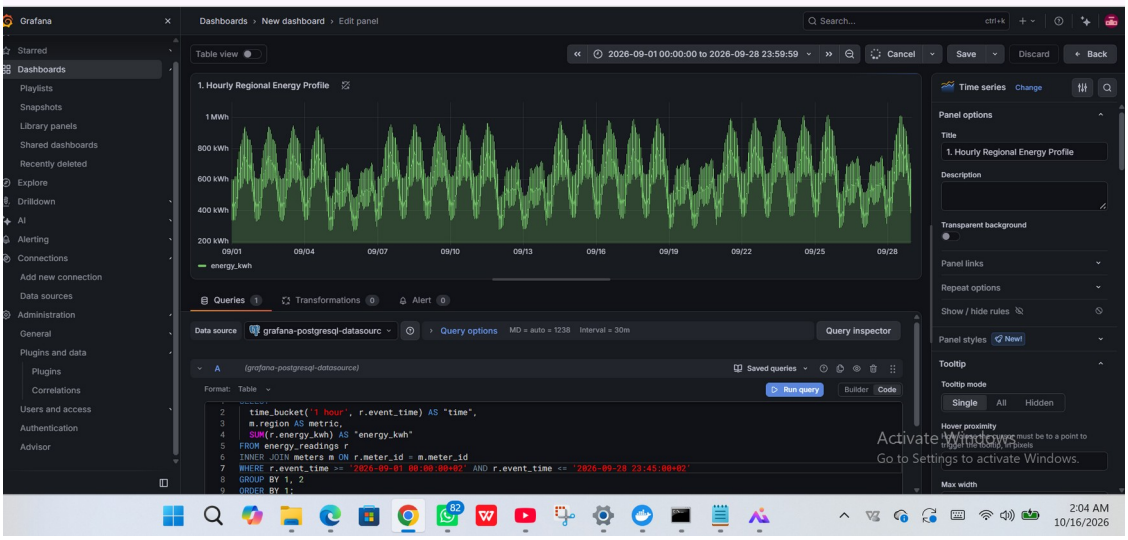


Figure 7.1-a: panel 1, hourly regional energy (kWh), Raw dashboard (2:04 AM, 16 October 2026)

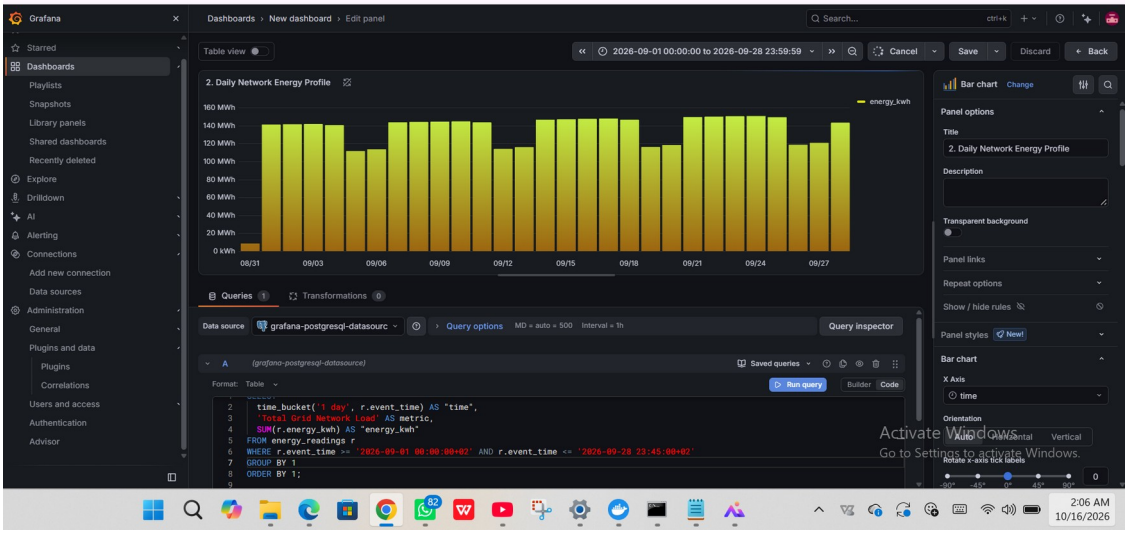


Figure 7.1-b: panel 2, daily network energy, Raw dashboard (2:06 AM, 16 October 2026)

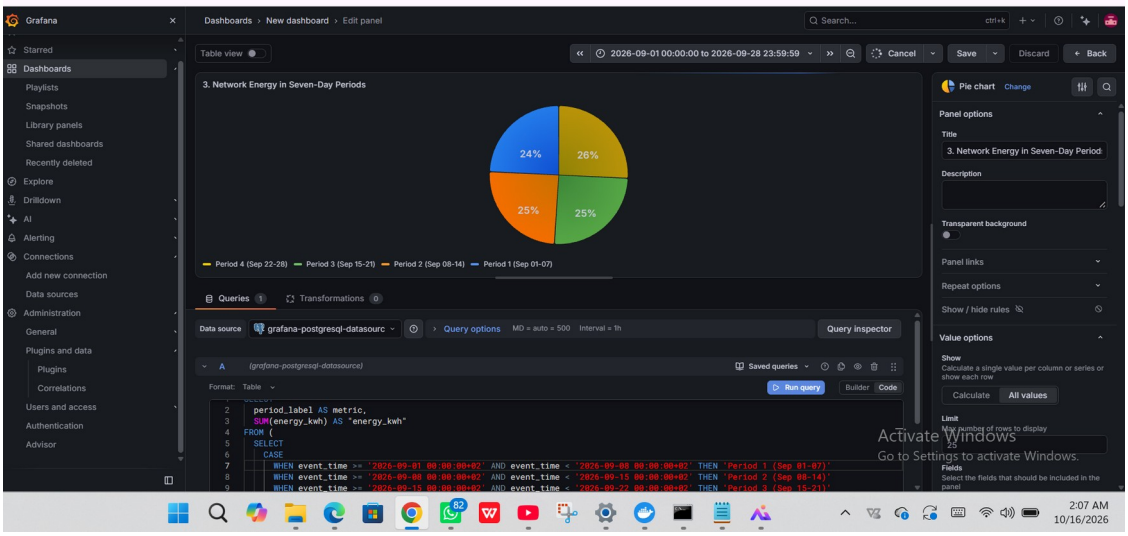


Figure 7.1-c: panel 3, network energy in the four seven-day periods, Raw dashboard (2:07 AM, 16 October 2026)

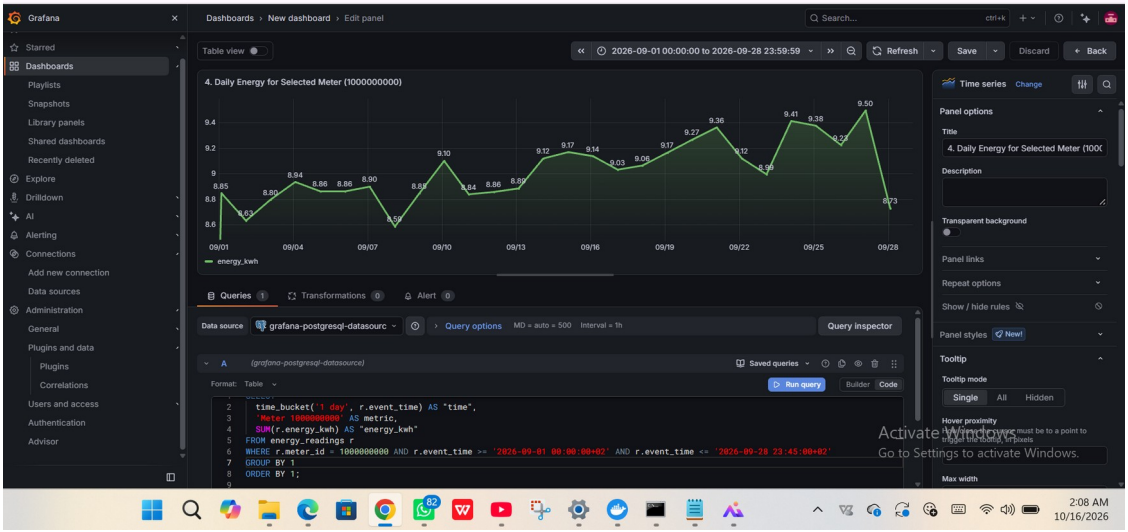


Figure 7.1-d: panel 4, daily energy for meter 1000000000, Raw dashboard (2:08 AM, 16 October 2026)

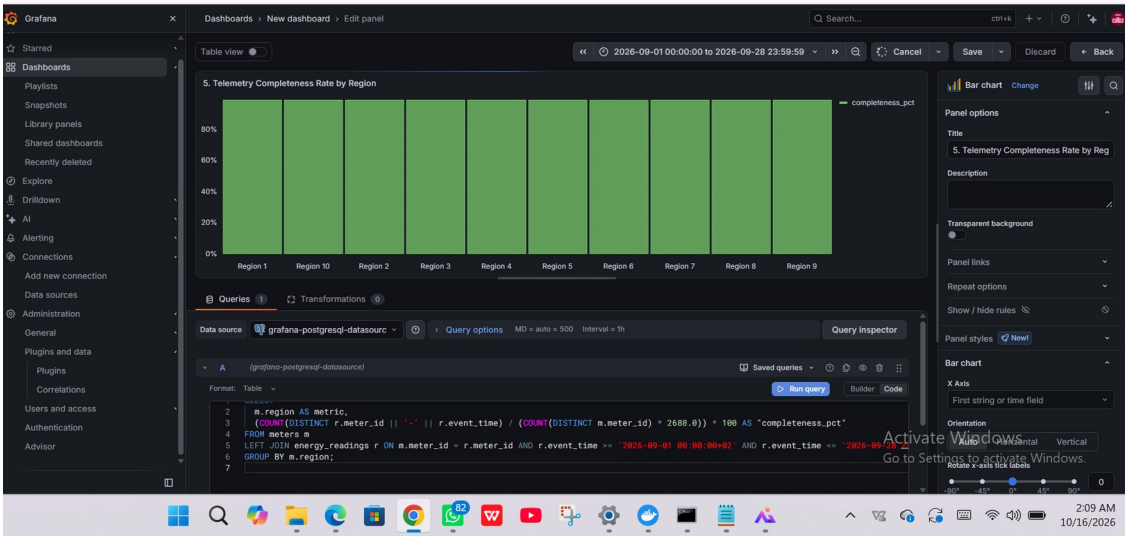


Figure 7.1-e: panel 5, telemetry completeness by region (%), Raw dashboard (2:09 AM, 16 October 2026)

Raw panel queries (as visible in the screenshots)

```
-- Panel 1
SELECT time_bucket('1 hour', r.event_time) AS "time", m.region AS metric, SUM(r.energy_kwh)
AS "energy_kwh"
```

```
FROM energy_readings r INNER JOIN meters m ON r.meter_id = m.meter_id
WHERE r.event_time >= '2026-09-01 00:00:00+02' AND r.event_time <= '2026-09-28 23:45:00+02'
GROUP BY 1, 2 ORDER BY 1;

-- Panel 2
SELECT time_bucket('1 day', r.event_time) AS "time", 'Total Grid Network Load' AS metric,
SUM(r.energy_kwh) AS "energy_kwh"
FROM energy_readings r WHERE r.event_time >= '2026-09-01 00:00:00+02' AND r.event_time <=
'2026-09-28 23:45:00+02'
GROUP BY 1 ORDER BY 1;

-- Panel 4
SELECT time_bucket('1 day', r.event_time) AS "time", 'Meter 1000000000' AS metric,
SUM(r.energy_kwh) AS "energy_kwh"
FROM energy_readings r WHERE r.meter_id = 1000000000 AND r.event_time >= '2026-09-01
00:00:00+02'
  AND r.event_time <= '2026-09-28 23:45:00+02' GROUP BY 1 ORDER BY 1;

-- Panel 5
SELECT m.region AS metric,
  (COUNT(DISTINCT r.meter_id || '-' || r.event_time) / (COUNT(DISTINCT m.meter_id) *
2688.0)) * 100 AS "completeness_pct"
FROM meters m LEFT JOIN energy_readings r ON m.meter_id = r.meter_id AND r.event_time >=
'2026-09-01 00:00:00+02' ...
GROUP BY m.region;
-- Panel 3: CASE on event_time into Period 1 to Period 4 labels, SUM(energy_kwh), grouped
by period label.
```

> **Limitations of the dashboards as built.** *(1) The panel queries use fixed dates (1 to 28 September) and a fixed planned count of 2,688 readings per meter instead of dashboard-level date and meter variables. (2) The daily panels use time_bucket('1 day') aligned to UTC midnight, which produces a small first bar on 31 August; CAT-aligned daily totals are in Section 5.1. (3) The hourly regional panel is displayed as a single series. (4) The figures are edit-mode screenshots of the panels.*

7.2 Build the Aggregate dashboard

The Aggregate dashboard has the same five panels. Hourly regional energy comes from grid_agg_hourly, daily and seven-day network energy and observed completeness counts from grid_agg_15min, and the meter history from grid_agg_daily. Stored energy sums and reading counts are summed when buckets are combined.

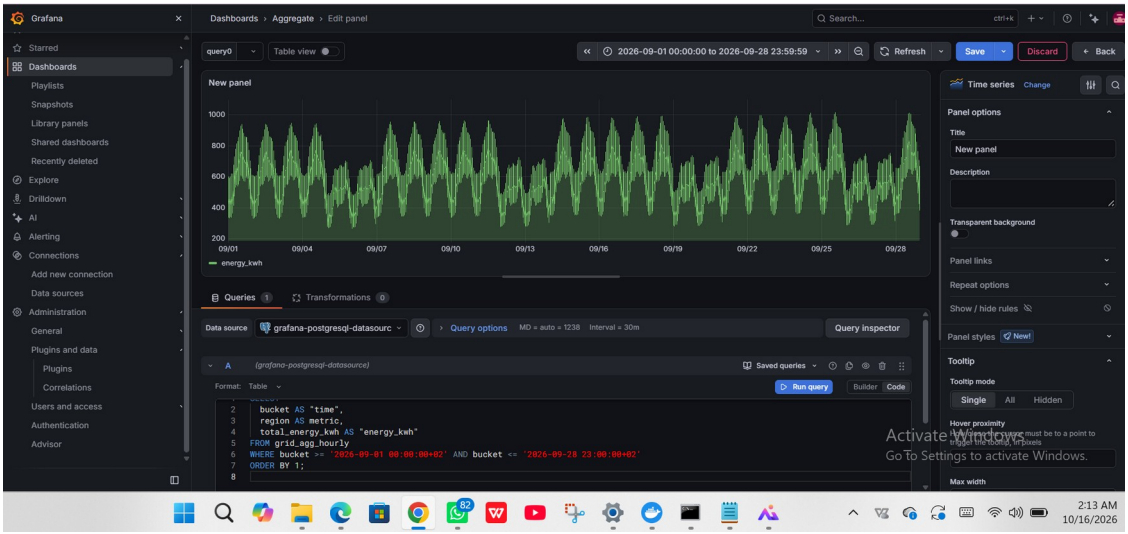


Figure 7.2-a: panel 1, hourly regional energy, Aggregate dashboard (2:13 AM, 16 October 2026)

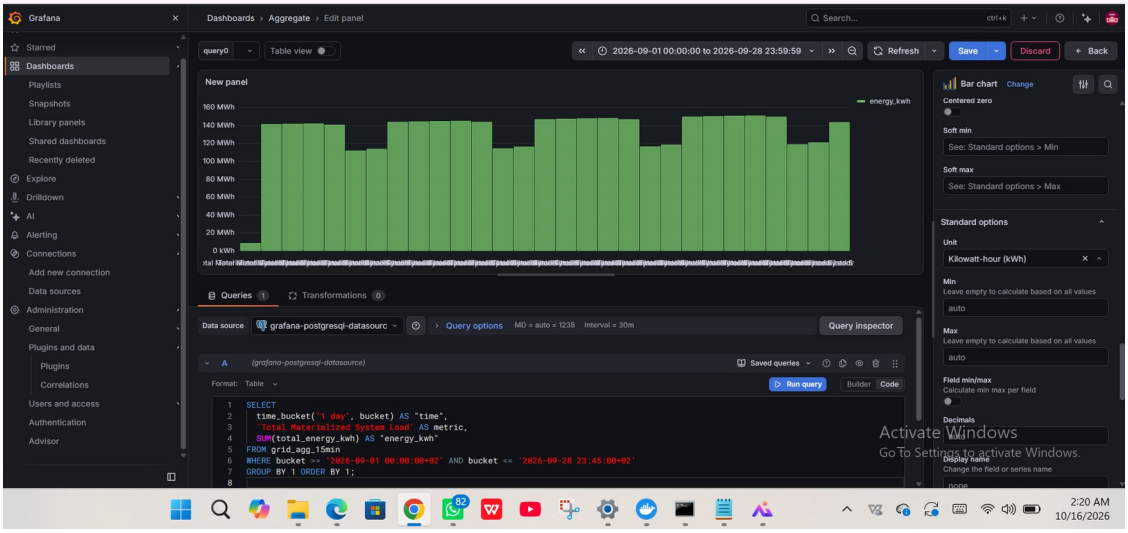


Figure 7.2-b: panel 2, daily network energy, Aggregate dashboard (2:20 AM, 16 October 2026)

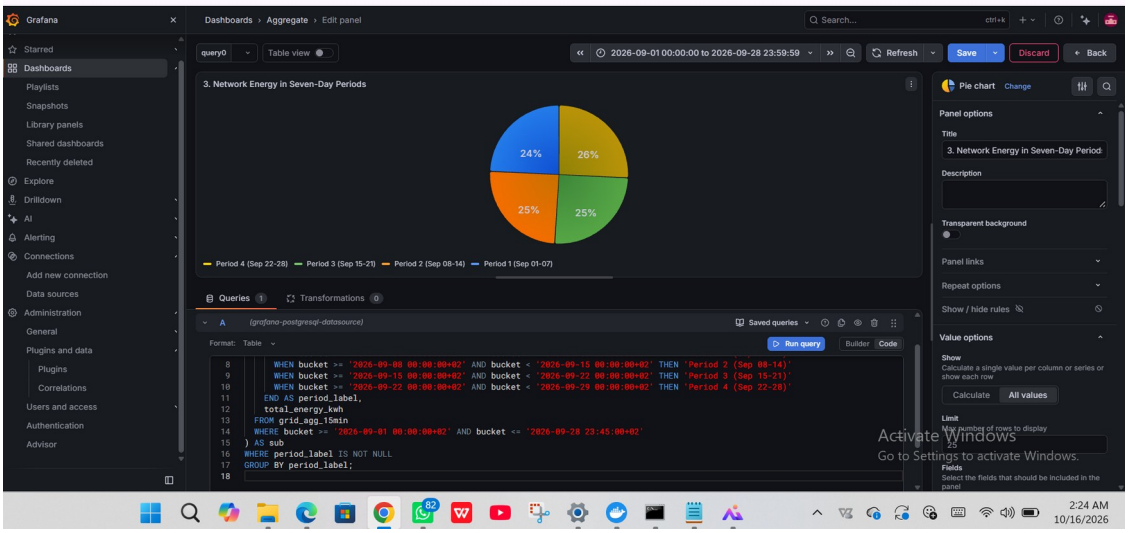


Figure 7.2-c: panel 3, seven-day periods, Aggregate dashboard (2:24 AM, 16 October 2026)

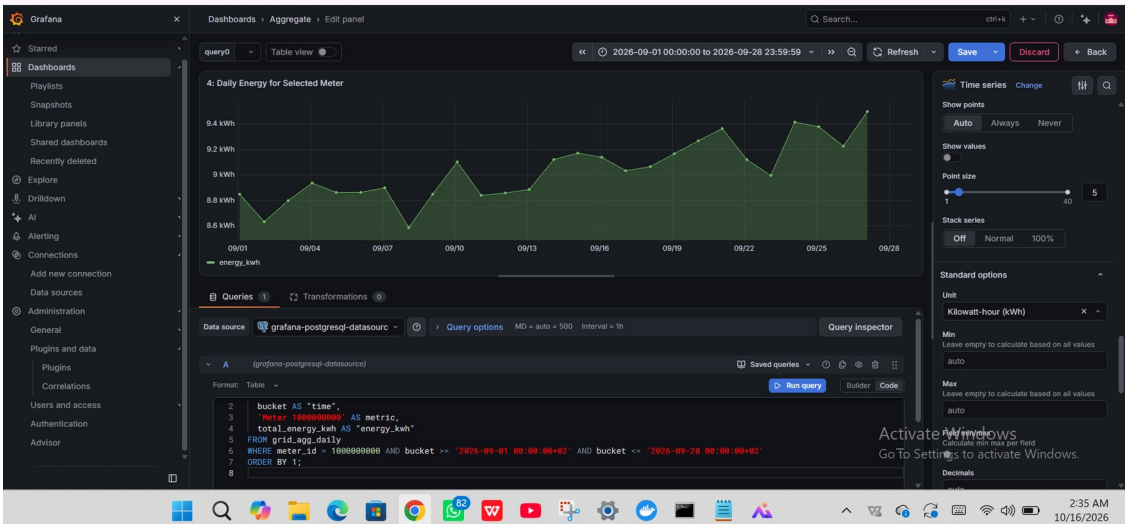


Figure 7.2-d: panel 4, daily energy for meter 1000000000, Aggregate dashboard (2:35 AM, 16 October 2026)

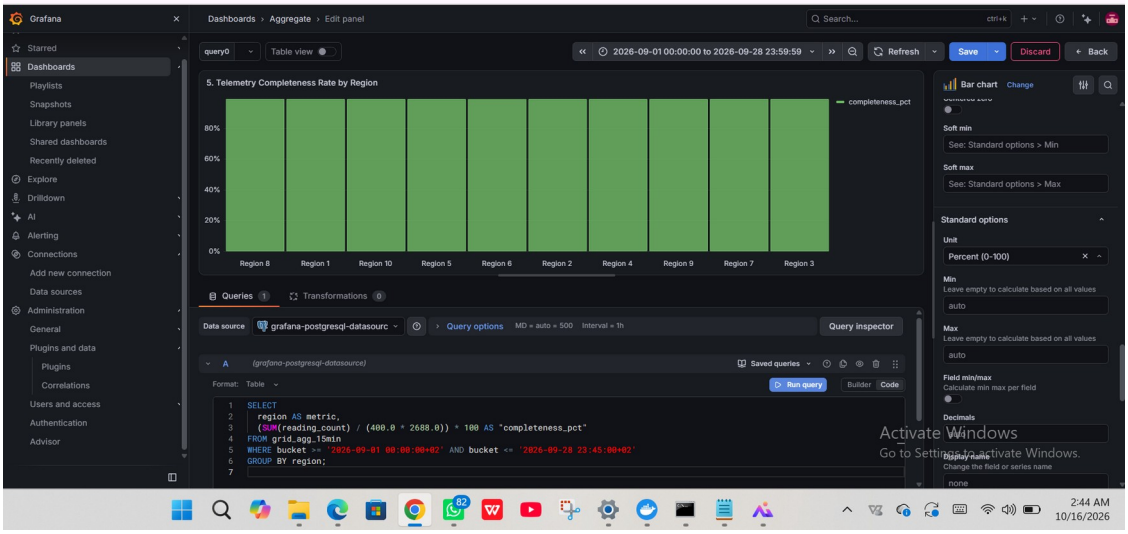


Figure 7.2-e: panel 5, completeness by region from stored reading counts, Aggregate dashboard (2:44 AM, 16 October 2026)

Aggregate panel queries (as visible in the screenshots)

```
-- Panel 1
SELECT bucket AS "time", region AS metric, total_energy_kwh AS "energy_kwh"
FROM grid_agg_hourly WHERE bucket >= '2026-09-01 00:00:00+02' AND bucket <= '2026-09-28
23:00:00+02' ORDER BY 1;

-- Panel 2
SELECT time_bucket('1 day', bucket) AS "time", 'Total Materialized System Load' AS metric,
SUM(total_energy_kwh) AS "energy_kwh"
FROM grid_agg_15min WHERE bucket >= '2026-09-01 00:00:00+02' AND bucket <= '2026-09-28
23:45:00+02' GROUP BY 1 ORDER BY 1;

-- Panel 3: CASE on bucket from grid_agg_15min into Period 1 to Period 4,
SUM(total_energy_kwh), GROUP BY period_label.

-- Panel 4
SELECT bucket AS "time", 'Meter 1000000000' AS metric, total_energy_kwh AS "energy_kwh"
FROM grid_agg_daily WHERE meter_id = 1000000000 AND bucket >= '2026-09-01 00:00:00+02' AND
bucket <= '2026-09-28 00:00:00+02' ORDER BY 1;

-- Panel 5
SELECT region AS metric, (SUM(reading_count) / (400.0 * 2688.0)) * 100 AS
"completeness_pct"
FROM grid_agg_15min WHERE bucket >= '2026-09-01 00:00:00+02' AND bucket <= '2026-09-28
23:45:00+02' GROUP BY region;
```

> **Note.** *The Aggregate dashboard shares these limitations (fixed dates, fixed planned count, UTC day buckets). Two panels were unsaved in the edit-mode screenshots (default title "New panel"), and the x-axis labels of the daily panel are garbled in Figure 7.2-b.*

7.3 Verify matching results

The brief sets both dashboards to 00:00 on 22 September through 00:00 on 29 September (exclusive), all regions and meter 1000000000, and requires identical keys and reading counts, energy differences of at most 0.001 kWh and completeness differences of at most 0.000001 percentage points.

The numerical panel-by-panel comparison for this period was not completed in this submission, so no equivalence between the Raw and Aggregate dashboards is claimed. The dashboards as built use fixed dates and UTC-aligned daily buckets (Section 7.1), so they would first need date variables and

CAT-aligned buckets before such a check could be run. The only comparison visible in the evidence is that the seven-day pie charts of both dashboards show the same shares (24%, 25%, 25% and 26%; Figures 7.1-c and 7.2-c), which is consistent with matching results but is not a numerical check.

7.4 Measure performance and judge the tradeoff

The group's notes report a whole-dashboard refresh of **4,492.8 ms for the Raw dashboard and 485.3 ms for the Aggregate dashboard**, a 9.3 times speed-up (4,492.8 / 485.3 = 9.26). The extra work for the aggregates is the initial manual refresh of 43.69 s + 24.19 s + 3.90 s = **71.78 s**, a background refresh every 15 minutes, and the storage of the three views (not measured, see Section 6.1).

Measure	Raw dashboard	Aggregate dashboard	Ratio
Whole-dashboard refresh (reported)	4,492.8 ms	485.3 ms	9.26×
Initial refresh of the three views (Section 6.1)	none	71.78 s	one-off

Judgement. If the reported figures hold, a 9.3-fold faster refresh is large for a control-room dashboard that refreshes repeatedly, and the cost is a one-off 71.78 s of refresh work, a small amount of extra storage (three views against a 1.4 GB raw table) and a background refresh every 15 minutes. The trade-off is staleness: with real-time aggregation off (Section 6.2), recent or late data is not visible until the next refresh. The speed-up therefore justifies the aggregates for monitoring dashboards, provided the three-day refresh lookback is kept.

> **Note.** *Per-panel query durations and the individual refresh records (one warm-up and ten measured refreshes per mode) were not captured in this submission; only the whole-dashboard figures above are available.*

8 Forecast the next day

The forecast target is hourly regional consumption in kWh. It is an offline forecast of observed consumption: completed historical hourly totals are treated as final, and the remaining missing readings are a limitation.

8.1 Prepare chronological data and features

src/prepare_forecast_data.py builds one row per region and CAT hour by summing energy_kwh from the cleaned readings: 28 days × 24 hours × 10 regions = **6,720 rows**. The query returned all 6,720 region-hours, so there were no absent hours and no predictor values to fill. Observed totals are kept as evaluation targets. Every target hour uses 00:00 at the start of its day as the forecast origin.

Feature	Lag or window	Earliest usable target timestamp
target_day_of_week (calendar)	none	2026-09-01 00:00
target_hour_of_day (calendar)	none	2026-09-01 00:00
lag_24h_energy (same hour, previous day)	24 hours	2026-09-03 00:00 as reported (retained rows require the 48-hour lookback)
lag_48h_energy (same hour, two days earlier)	48 hours	2026-09-03 00:00

Split	Target dates	Rows before lookback exclusion	Excluded	Retained rows
Training	1 to 14 September	3,360	480	2,880 (3 to 14 September)
Validation	15 to 21 September	1,680	0	1,680
Test	22 to 28 September	1,680	0	1,680
Total	1 to 28 September	6,720	480	6,240

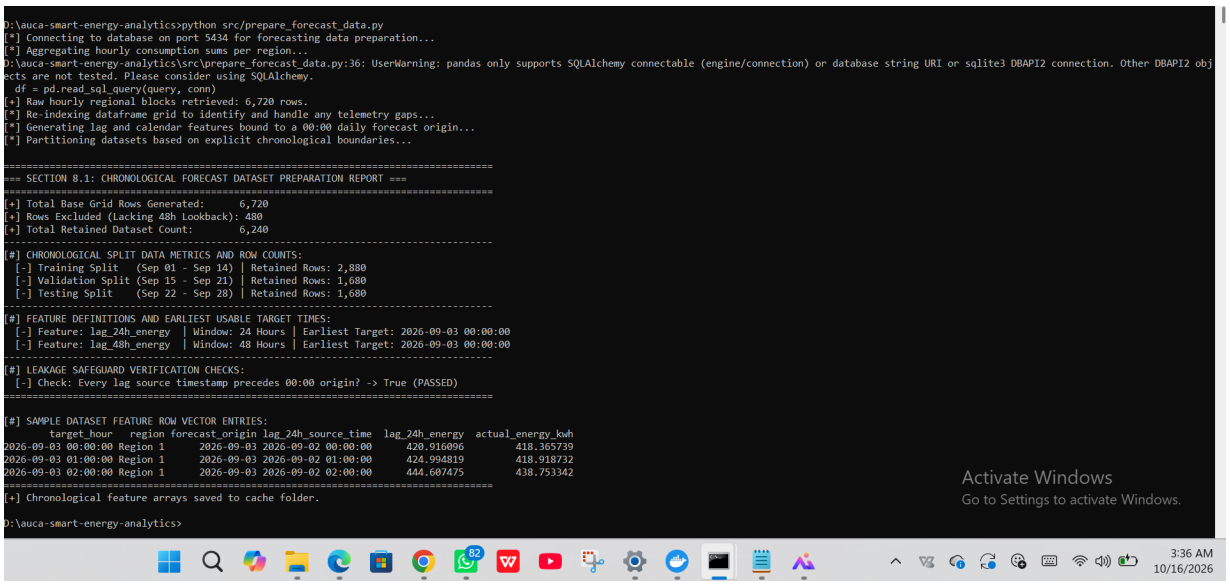


Figure 8.1-a: dataset preparation, split sizes, features, leakage check and sample rows (3:36 AM, 16 October 2026)

The data is split by target timestamp, not at random. The 480 excluded training rows are the first 48 hours (1 and 2 September) for all ten regions, which lack the 48-hour lookback. The same usable training rows are used for every model configuration. The StandardScaler is fitted on the training split only. The automated check reports that **every lag source timestamp precedes its 00:00 forecast origin: True (passed)**. The sample rows show, for example, the target 2026-09-03 00:00 for Region 1 using a lag source of 2026-09-02 00:00.

Sample rows of the forecast dataset

Target hour	Region	Forecast origin	Lag source time	lag_24h_energy (kWh)	Actual (kWh)
2026-09-03 00:00	Region 1	2026-09-03 00:00	2026-09-02 00:00	420.916096	418.365739
2026-09-03 01:00	Region 1	2026-09-03 00:00	2026-09-02 01:00	424.994819	418.918732
2026-09-03 02:00	Region 1	2026-09-03 00:00	2026-09-02 02:00	444.607475	438.753342

In each row the lag source time (2 September) is earlier than the forecast origin (3 September 00:00), which is what the leakage check verifies for all rows. The previous-day value differs from the actual by 2.55, 6.08 and 5.85 kWh in these three rows.

> **Note.** *Reading counts per region-hour are not included in the dataset report, and no region-hour was absent (6,720 of 6,720 present). The feature lag_24h alone is usable from 2 September; 3 September applies because the longest lookback (48 hours) sets the common retained rows.*

Worked example of the error metrics

Prediction error is predicted minus actual. The first three rows of the baseline sample in Figure 8.2-a (Region 1, 22 September, 00:00 to 02:00) give the following errors.

Target hour	Actual (kWh)	Baseline prediction	Error	Absolute error	Squared error
00:00	437.3753	432.5103	−4.8650	4.8650	23.67
01:00	447.8441	443.2056	−4.6385	4.6385	21.52
02:00	463.1968	468.3796	+5.1828	5.1828	26.86

For these three rows MAE = (4.8650 + 4.6385 + 5.1828) / 3 = 4.896 kWh and RMSE = √((23.67 + 21.52 + 26.86) / 3) = 4.90 kWh. Over all 1,680 test rows the baseline has MAE 42.92 kWh and RMSE 79.44 kWh. RMSE is larger than MAE because squaring gives more weight to large errors, such as the

weekend misses of several hundred kWh in Figure 8.4-b. The ratio RMSE/MAE is 1.85 for the baseline and 1.66 for the model, which says the baseline's errors are more concentrated in a few large misses.

The same three hours for the frozen model (Figure 8.3-a) give predictions of 445.2079, 446.8112 and 468.7873 kWh against actuals of 437.3753, 447.8441 and 463.1968 kWh, so errors of +7.83, −1.03 and +5.59 kWh. On these three hours the model is no better than the baseline (MAE 4.82 kWh against 4.90 kWh); its advantage comes from the weekend and Monday days where the baseline fails. A small sample must not be read as the overall result.

8.2 Evaluate the previous-day baseline

At 00:00 CAT on each test day, the baseline forecasts the 24 hourly values of that day with the previous day's corresponding hour, for all ten regions (src/evaluate_baseline.py). That gives 7 days × 24 hours × 10 regions = **1,680 predictions**.

Baseline metric (1,680 test predictions)	Value
MAE	42.9159 kWh
RMSE	79.4397 kWh

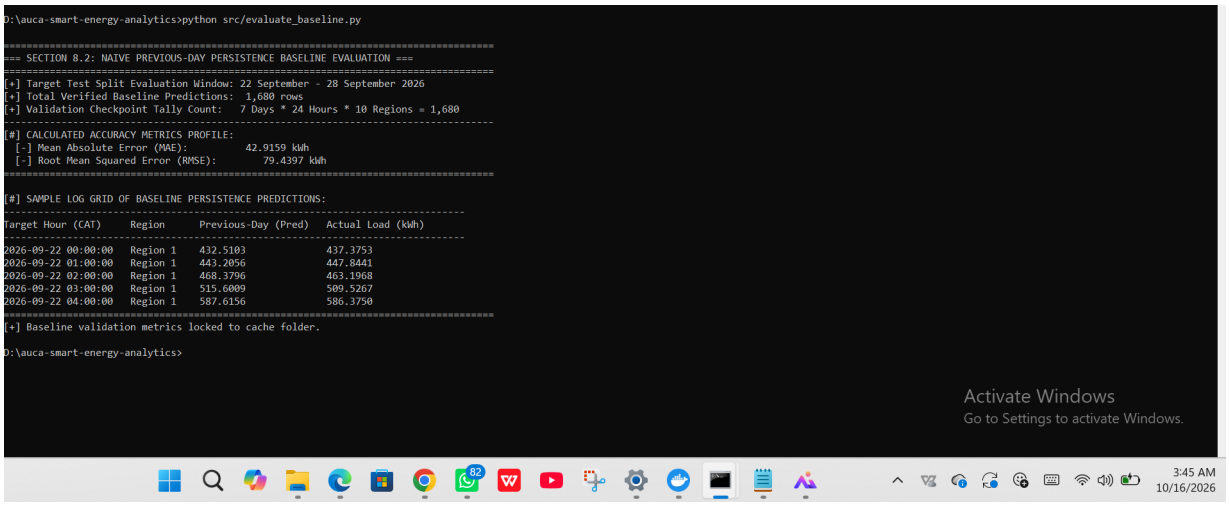


Figure 8.2-a: baseline run with prediction count, metrics and sample predictions (3:45 AM, 16 October 2026)

8.3 Train and evaluate one regression model

The model is a **LightGBM gradient-boosted regressor**, trained in src/evaluate_regression.py. Both configurations use the shared predictors target_day_of_week, target_hour_of_day, lag_24h_energy and lag_48h_energy, and the StandardScaler fitted on the training rows only.

Config ID	Model settings	Training rows	Validation rows	Validation MAE (kWh)	Validation RMSE (kWh)
Config_1 (shallow)	n_estimators = 50, max_depth = 3	2,880	1,680	26.3714	37.0512
Config_2 (deep) (selected)	n_estimators = 150, max_depth = 6	2,880	1,680	7.7102	12.2643

Config_2 has the lowest validation MAE and was frozen, with its preprocessing, before testing. The frozen model generated the same **1,680 test predictions** as the baseline, using only information available before each daily forecast origin; earlier completed test days supply later lag values but were not used to refit the model or the preprocessing.

Frozen model on the test split	Value
Predictions	1,680

Frozen model on the test split	Value
MAE	8.8464 kWh
RMSE	14.6431 kWh

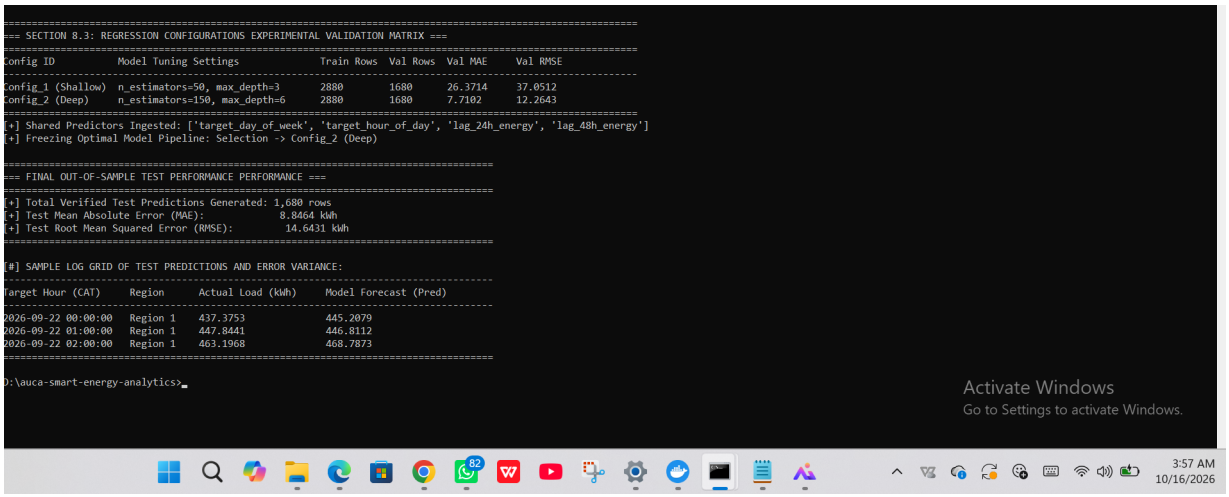


Figure 8.3-a: validation comparison, selected configuration, test metrics and sample predictions (3:57 AM, 16 October 2026)

Observation. The deeper model reduces validation MAE from 26.37 to 7.71 kWh. The test MAE (8.85 kWh) is slightly higher than the validation MAE (7.71 kWh), as expected for data the model has not seen.

8.4 Judge forecast usefulness

Prediction error is predicted minus actual kWh; MAE averages the absolute values and RMSE is the square root of the mean squared error, both in kWh. The horizon is 24 hourly predictions per region per test day.

Scope	Baseline MAE	Baseline RMSE	Model MAE	Model RMSE	MAE reduction
All 1,680 observations	42.9159	79.4397	8.8464	14.6431	79.4%
Region 1	45.0402	83.7197	9.7957	17.8296	78.3%
Region 2	41.8940	76.8502	5.6643	7.3377	86.5%
Region 3	42.9694	77.6414	7.2438	8.9020	83.1%
Region 4	35.0956	64.2745	6.4581	8.1659	81.6%
Region 5	44.6148	81.1315	11.1544	16.1840	75.0%
Region 6	39.7013	74.8128	6.4620	8.2737	83.7%
Region 7	39.9265	74.0009	6.1123	7.8801	84.7%
Region 8	56.0162	102.0636	21.2788	32.1405	62.0%
Region 9	43.9344	80.2066	8.1448	11.4932	81.5%
Region 10	39.9661	74.3429	6.1502	8.3043	84.6%

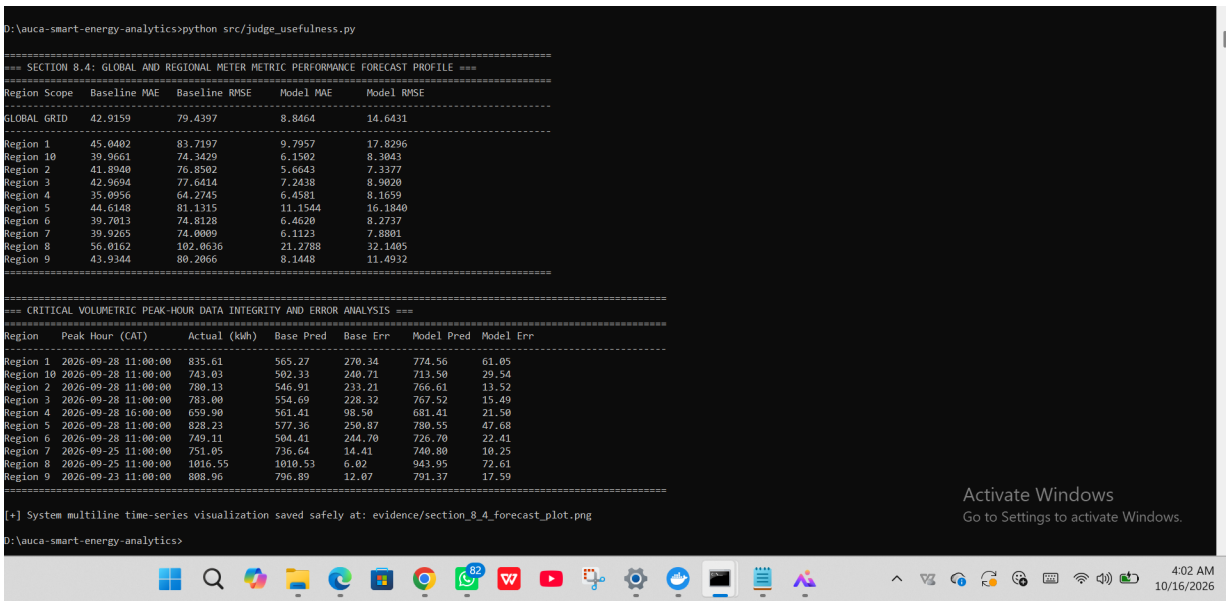


Figure 8.4-a: global and regional metrics and peak-hour errors (4:02 AM, 16 October 2026)

Highest actual consumption hour per region in the test period

Region	Peak hour (CAT)	Actual (kWh)	Baseline prediction	Baseline abs. error	Model prediction	Model abs. error
Region 1	28 Sep 11:00	835.61	565.27	270.34	774.56	61.05
Region 2	28 Sep 11:00	780.13	546.91	233.21	766.61	13.52
Region 3	28 Sep 11:00	783.00	554.69	228.32	767.52	15.49
Region 4	28 Sep 16:00	659.90	561.41	98.50	681.41	21.50
Region 5	28 Sep 11:00	828.23	577.36	250.87	780.55	47.68
Region 6	28 Sep 11:00	749.11	504.41	244.70	726.70	22.41
Region 7	25 Sep 11:00	751.05	736.64	14.41	740.80	10.25
Region 8	25 Sep 11:00	1,016.55	1,010.53	6.02	943.95	72.61
Region 9	23 Sep 11:00	808.96	796.89	12.07	791.37	17.59
Region 10	28 Sep 11:00	743.03	502.33	240.71	713.50	29.54

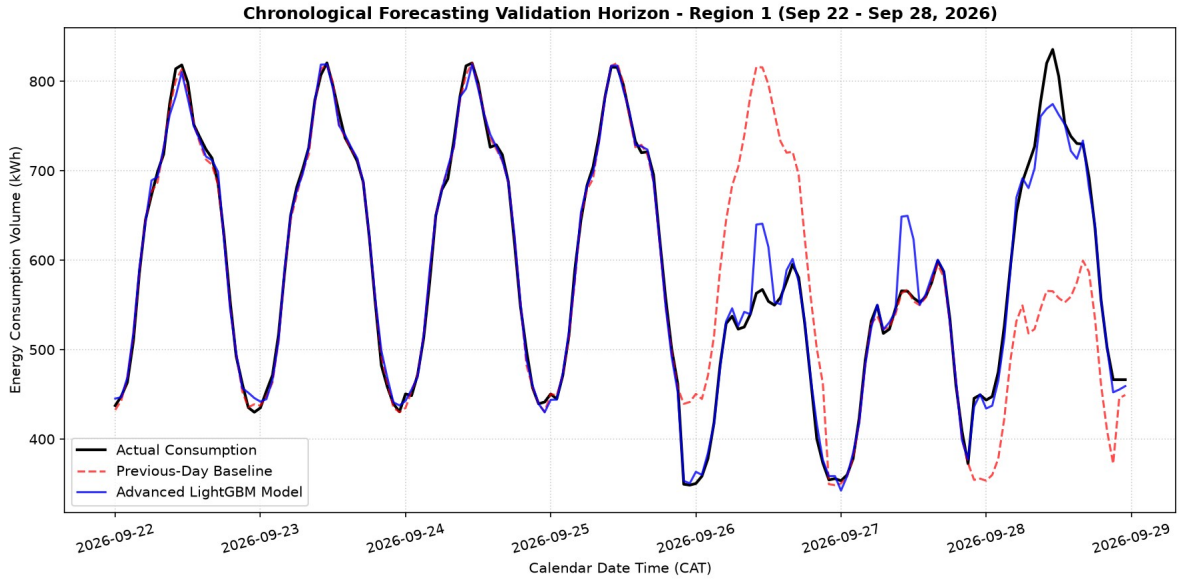


Figure 8.4-b: actual, baseline and model consumption for Region 1 over 22 to 28 September (the plot title reads "Validation Horizon" but it covers the test days)

Does the model improve on the baseline? Yes. Over all 1,680 test observations MAE falls from 42.92 to 8.85 kWh (79.4% lower) and RMSE from 79.44 to 14.64 kWh (81.6% lower). The model has the lower MAE and RMSE in every one of the ten regions; the smallest MAE reduction is 62.0% (Region 8) and the largest is 86.5% (Region 2). The Region 1 plot shows why: on the weekend days (26 and 27 September) and on Monday 28 September the baseline repeats the previous day's pattern and misses by hundreds of kWh, while the model, which has day-of-week information, follows the actual curve.

Are the errors useful for planning? For day-ahead planning, probably yes. Relative to the mean regional hourly consumption of about 573.5 kWh (3,853,704.35 kWh over 6,720 region-hours), the model MAE of 8.85 kWh is about 1.5% and the baseline MAE of 42.92 kWh is about 7.5%. **Peak hours are the weak point.** At the highest-consumption hour of each region, the model has the smaller error in eight of ten regions, but it underpredicts the peak in nine of ten (only Region 4 is overpredicted); the baseline is better in Region 8 (6.02 vs 72.61 kWh) and Region 9 (12.07 vs 17.59 kWh). Peak errors for the model are 10 to 73 kWh, about 7% of actual in the worst cases (Region 1 and Region 8). Region 8 also has the largest error overall (MAE 21.28 kWh), so planners should keep a safety margin at peaks. The plot also shows model overshoots around midday on 26 and 27 September (about 640 to 650 kWh against about 565 kWh actual).

Missing readings and delayed arrivals. The targets are observed sums, so they understate true consumption by about 1% missing readings (completeness 98.993%), and the model learns that slightly low level. In live use at 00:00, the lag features for the last hours of the previous day may still be incomplete because delayed readings arrive 30 to 180 minutes late (Section 5.3 showed the effect on a daily total is small, 0.10%, but it falls on exactly those last hours), and missing readings would leave gaps in the lag inputs. The offline evaluation treats completed hourly totals as final and so does not include these effects. A live deployment would need rules for incomplete hours and a periodic restatement of forecasts.

9 Reproducibility and contributions

9.1 Report organisation

This report follows the subsection order of the brief, with each screenshot placed inside the subsection it supports and earlier figures referred to again where a later subsection asks only for interpretation.

9.2 Make the work reproducible

The GitHub repository and the ZIP archive contain the same final files: the unchanged smart_meter_simulator.py, the capture and ingestion programs, the SQL files, the analysis and model code, recorded benchmark timings, the validation comparison, dashboard queries and configuration, recorded environment details and a README with the execution order. The full generated dataset stays local and is regenerated and loaded by the programs below. Passwords and connection secrets are placeholders.

GitHub repository: https://github.com/ericniyitanga082/Big-Data-Analytics-Project1

Execution order (from the project scripts)

Step	Command	Purpose
0	docker run -d --name auca_emqx_container -p 1883:1883 -p 18083:18083 emqx/emqx:5.3.0	1.1 broker
0	docker run -d --name auca_timescaledb_container -p 5434:5432 -e POSTGRES_PASSWORD=<password> timescale/timescaledb:latest-pg15	1.1 database (replace <password> with your own)
1	psql: create tables, hypertable, extension (sql/*.sql)	1.2
2	python src/populate_meters.py	1.2 meters (4,000 rows)
3	python src/consume_pilot.py (start first), then python src/publish_pilot.py	1.3 pilot (four counts)

Step	Command	Purpose
4	python src/bulk_load_historical.py	2.1 full load
5	python src/investigate_quality.py	2.2
6	python src/clean_benchmark.py	2.3
7	python src/compare_groups.py	3.1 (and 3.2 benchmark script)
8	python src/run_compression_experiment.py; python src/compression_metrics.py; python src/compression_cost_analysis.py; python src/run_all_compressed_benchmarks.py	4.1 to 4.3
9	python src/demand_patterns.py; regional_normalization.py; reporting_quality.py; detect_anomalies.py	5.1 to 5.4
10	python src/continuous_aggregates.py; python src/test_freshness.py	6.1, 6.2
11	Grafana dashboards Raw and Aggregate (JSON export in repository)	7
12	python src/prepare_forecast_data.py; evaluate_baseline.py; evaluate_regression.py; judge_usefulness.py	8

9.3 Document contributions

Member	Student ID	Role	Individual contribution
BIZIMUNGU Theoneste	101398	Database and ingestion lead	Docker setup of EMQX and TimescaleDB, schema and hypertable creation, meters population, pilot publisher and subscriber, full historical load, data-quality audit and cleaning (Sections 1 and 2).
NIYITANGA Eric	101393	Analytics, forecasting and repository lead	Demand patterns, regional and customer analysis, reporting quality, anomaly detection, forecasting pipeline with baseline, LightGBM and evaluation, GitHub repository and report assembly (Sections 5, 8 and 9).
NDASHIMYE Janvier	101363	Storage, aggregates and dashboards lead	Chunk-interval comparison, compression experiments, continuous aggregates and freshness test, Raw and Aggregate Grafana dashboards (Sections 3, 4, 6 and 7).

All members should be prepared to explain and present the work if called upon.

Appendix A: Supporting SQL queries

The queries below reproduce the checks referred to in the report and can be run in psql connected to auca_energy_db. They use standard TimescaleDB 2.x views; column names can differ slightly between versions.

A.1 Environment and settings (1.1)

```
SELECT version();
SELECT extname, extversion FROM pg_extension WHERE extname = 'timescaledb';
SHOW timezone;
SHOW shared_buffers;
SHOW work_mem;
SHOW max_parallel_workers_per_gather;
SHOW jit;
-- set the database session time zone if SHOW timezone is not Africa/Kigali:
ALTER DATABASE auca_energy_db SET timezone TO 'Africa/Kigali';
```

A.2 Table structure, chunk intervals, indexes and chunks (1.2, 3.1)

```
\d energy_live
\d energy_readings
\d meters
SELECT count(*) FROM meters;

SELECT hypertable_name, column_name, time_interval
FROM timescaledb_information.dimensions
WHERE hypertable_name LIKE 'energy_%' ORDER BY hypertable_name;

SELECT tablename, indexname, indexdef
FROM pg_indexes
WHERE tablename IN ('energy_readings','energy_readings_3h','energy_readings_week')
ORDER BY tablename, indexname;

SELECT hypertable_name, count(*) AS chunks, min(range_start) AS first_start, max(range_end)
AS last_end
FROM timescaledb_information.chunks
WHERE hypertable_name LIKE 'energy_readings%'
GROUP BY hypertable_name ORDER BY hypertable_name;
```

A.3 Pilot stream counts (1.3)

```
SELECT count(*) AS stored_rows FROM energy_live;
SELECT * FROM energy_live ORDER BY event_time, meter_id LIMIT 5;
-- deliberate duplicates and redelivery both show up here:
SELECT count(*) - count(DISTINCT (meter_id, event_time)) AS extra_copies FROM energy_live;
```

A.4 Raw load checks (2.1)

```
SELECT count(*) AS rows, count(DISTINCT meter_id) AS meters,
       min(event_time) AS earliest, max(event_time) AS latest
FROM energy_readings;

SELECT pg_size_pretty(table_bytes) AS table_size, pg_size_pretty(index_bytes) AS
index_size,
       pg_size_pretty(toast_bytes) AS toast_size, pg_size_pretty(total_bytes) AS total_size
FROM hypertable_detailed_size('energy_readings');
```

A.5 Cleaning and query checks (2.3, 3.2)

```
-- no duplicate pairs remain
SELECT count(*) AS duplicate_pairs FROM (
  SELECT meter_id, event_time FROM energy_readings GROUP BY 1, 2 HAVING count(*) > 1) d;
```

```
-- Q2 with ties kept: all intervals whose total is at least the tenth-highest total
WITH t AS (SELECT event_time, SUM(power_kw) AS total FROM energy_readings
           WHERE event_time >= '2026-09-22 00:00:00+02' AND event_time < '2026-09-29
00:00:00+02'
           GROUP BY event_time),
     r AS (SELECT *, rank() OVER (ORDER BY total DESC) AS rk FROM t)
SELECT event_time, total FROM r WHERE rk <= 10 ORDER BY total DESC, event_time;

-- Q3 returns one row per meter
SELECT count(*) FROM (SELECT meter_id, SUM(energy_kwh) FROM energy_readings GROUP BY
meter_id) q;
```

A.6 Compression settings, sizes, chunk lists and policies (4.1 to 4.3)

```
SELECT hypertable_name, attname, segmentby_column_index, orderby_column_index, orderby_asc
FROM timescaledb_information.compression_settings
WHERE hypertable_name IN ('energy_time_compressed','energy_meter_compressed')
ORDER BY hypertable_name, attname;

SELECT chunk_name, range_start, range_end, is_compressed
FROM timescaledb_information.chunks
WHERE hypertable_name = 'energy_time_compressed' ORDER BY range_start;   -- repeat for the
meter table

SELECT * FROM hypertable_compression_stats('energy_time_compressed');
SELECT * FROM hypertable_compression_stats('energy_meter_compressed');

-- size including TOAST, so data + index + toast adds up to total
SELECT 'energy_time_compressed' AS t, table_bytes, index_bytes, toast_bytes, total_bytes
FROM hypertable_detailed_size('energy_time_compressed');

-- rows held in compressed chunks (denominator 10,643,735)
SELECT count(*) AS rows_in_compressed_chunks,
       round(100.0 * count(*) / 10643735, 2) AS pct
FROM energy_readings
WHERE event_time < (SELECT max(range_end) FROM timescaledb_information.chunks
                    WHERE hypertable_name = 'energy_time_compressed' AND is_compressed);

SELECT job_id, hypertable_name, config, scheduled, next_start
FROM timescaledb_information.jobs
WHERE hypertable_name IN ('energy_time_compressed','energy_meter_compressed');
```

A.7 Region by customer type (5.2)

```
SELECT region, customer_type, count(*) AS meters
FROM meters GROUP BY region, customer_type ORDER BY region, customer_type;

SELECT m.region, m.customer_type, SUM(r.energy_kwh) AS energy_kwh
FROM energy_readings r JOIN meters m USING (meter_id)
GROUP BY 1, 2 ORDER BY 1, 2;
```

A.8 Continuous aggregate definitions, sizes and policies (6.1)

```
SELECT view_name, materialization_hypertable_schema, materialization_hypertable_name,
       materialized_only, view_definition
FROM timescaledb_information.continuous_aggregates;

-- size of each stored aggregate (use the names printed above)
SELECT * FROM hypertable_detailed_size('_timescaledb_internal._materialized_hypertable_N');
```

```
SELECT job_id, hypertable_name, schedule_interval, config, scheduled
FROM timescaledb_information.jobs
WHERE proc_name = 'policy_refresh_continuous_aggregate';
```

A.9 A per-meter anomaly rule to try as Rule B (5.4, proposal, not yet run)

The thresholds must be chosen from the distribution of the cleaned data, not tuned on the labelled events. The sketch below compares each reading with the same meter's median for the same time of day and weekday/weekend type, and flags readings far above or below it. Choose the two multipliers from the distribution of the ratio (for example a high percentile), and require several consecutive flagged intervals before reporting an event.

```
WITH tagged AS (
  SELECT meter_id, event_time, power_kw,
         (event_time AT TIME ZONE 'Africa/Kigali')::time AS tod,
         EXTRACT(ISODOW FROM event_time AT TIME ZONE 'Africa/Kigali') >= 6 AS weekend
  FROM energy_readings),
base AS (
  SELECT meter_id, tod, weekend, percentile_cont(0.5) WITHIN GROUP (ORDER BY power_kw) AS
med
  FROM tagged GROUP BY 1, 2, 3)
SELECT t.meter_id, t.event_time, t.power_kw, b.med, t.power_kw / b.med AS ratio
FROM tagged t JOIN base b USING (meter_id, tod, weekend)
WHERE t.power_kw > :high_multiplier * b.med OR t.power_kw < :low_fraction * b.med;
```

Appendix B: per-chunk compression durations

Values are read from Figure 4.1-a (compression log, 10:31 PM, 15 October 2026). Chunk end boundaries are in CAT; a boundary of 02:00 CAT is 00:00 UTC. The sums reproduce the compression cost reported in 4.2.

Chunk	Chunk end (CAT)	Time-only (s)	Meter-segmented (s)	Ratio
1	2026-09-01 02:00:00+02	0.157	0.655	4.2×
2	2026-09-02 02:00:00+02	1.035	2.284	2.2×
3	2026-09-03 02:00:00+02	0.830	1.636	2.0×
4	2026-09-04 02:00:00+02	0.768	2.110	2.7×
5	2026-09-05 02:00:00+02	0.741	1.625	2.2×
6	2026-09-06 02:00:00+02	0.836	2.855	3.4×
7	2026-09-07 02:00:00+02	1.168	2.305	2.0×
8	2026-09-08 02:00:00+02	0.864	2.147	2.5×
9	2026-09-09 02:00:00+02	0.828	2.238	2.7×
10	2026-09-10 02:00:00+02	0.719	2.710	3.8×
11	2026-09-11 02:00:00+02	1.654	1.813	1.1×
12	2026-09-12 02:00:00+02	0.817	2.091	2.6×
13	2026-09-13 02:00:00+02	2.125	2.411	1.1×
14	2026-09-14 02:00:00+02	1.500	3.340	2.2×
15	2026-09-15 02:00:00+02	1.884	3.037	1.6×
16	2026-09-16 02:00:00+02	1.571	1.966	1.3×
17	2026-09-17 02:00:00+02	1.363	1.700	1.2×
18	2026-09-18 02:00:00+02	1.524	1.757	1.2×
19	2026-09-19 02:00:00+02	1.208	2.037	1.7×
20	2026-09-20 02:00:00+02	1.196	1.514	1.3×
21	2026-09-21 02:00:00+02	0.894	1.341	1.5×
22	2026-09-22 02:00:00+02	0.636	1.548	2.4×
23	2026-09-23 02:00:00+02	0.915	1.774	1.9×
24	2026-09-24 02:00:00+02	0.873	1.525	1.7×
25	2026-09-25 02:00:00+02	0.637	1.982	3.1×
Total		26.743	50.401	1.88×

The meter-segmented compression is slower on every chunk, by 1.1 to 4.2 times, and the ratio is highest on the first chunk (0.655 s against 0.157 s), which holds only the first two hours of 1 September.

Appendix C: derived calculations

Every derived figure in the report is reproduced here from the measured values, so that a reader can check it.

Quantity	Calculation	Result
Missing percentage	100 × 108,265 / 10,752,000	1.0069%
Duplicate percentage	100 × 53,113 / 10,643,735	0.4990%
Delayed percentage	100 × 213,090 / 10,643,735	2.0020%
Completeness	100 × 10,643,735 / 10,752,000	98.9931%
Energy removed by cleaning	3,872,863.8932 − 3,853,704.3468	19,159.5464 kWh (0.495%)
Mean energy per removed row	19,159.5464 / 53,113	0.3607 kWh

Quantity	Calculation	Result
Mean energy per reading	3,853,704.3468 / 10,643,735	0.3621 kWh
Compression ratio, time-only	1,423.87 / 619.57	2.30×
Compression ratio, meter-segmented	1,423.87 / 617.10	2.31×
Percentage saved, time-only	100 × (1,423.87 − 619.57) / 1,423.87	56.49%
Percentage saved, meter-segmented	100 × (1,423.87 − 617.10) / 1,423.87	56.66%
Q3 penalty, 3 hours vs 1 week	31,034.41 / 437.09	71.0×
Q4 segmentation gain	26.82 / 2.12	12.7× (92.1% lower)
Q1 improvement, time-only (benchmark baseline)	100 × (167.35 − 50.19) / 167.35	70.0%
Q1 improvement, time-only (same-session baseline)	100 × (68.28 − 50.94) / 68.28	25.4%
Aggregate refresh total	43.69 + 24.19 + 3.90	71.78 s
Weekly growth, periods 1 to 2	100 × (953,714.97 − 934,199.06) / 934,199.06	+2.09%
Weekly growth, periods 2 to 3	100 × (973,230.03 − 953,714.97) / 953,714.97	+2.05%
Weekly growth, periods 3 to 4	100 × (992,560.29 − 973,230.03) / 973,230.03	+1.99%
Energy in the peak interval	7,993.69 kW × 0.25 h	1,998.4 kWh
Industrial share of energy	100 × 2,220,672.04 / 3,853,704.35	57.62%
Late-arrival effect, 21 September	100 × 152.1783 / 149,552.7229	0.1018%
False-flag rate outside events	100 × 80,109 / (10,643,735 − 480)	0.7527%
Meters affected by false flags	100 × 124 / 4,000	3.10%
MAE reduction (model vs baseline)	100 × (42.9159 − 8.8464) / 42.9159	79.39%
RMSE reduction (model vs baseline)	100 × (79.4397 − 14.6431) / 79.4397	81.57%
Mean regional hourly consumption	3,853,704.35 / 6,720	573.5 kWh
Relative MAE, model	8.8464 / 573.5	1.54%
Relative MAE, baseline	42.9159 / 573.5	7.48%

The denominator of the false-flag rate assumes 40 injected events of 12 intervals each (480 readings), as defined by the supplied generator.

Appendix D: limitations and threats to validity

- **Single machine, single run set.** All timings come from one Windows machine running PostgreSQL inside Docker, with a small number of repetitions. Differences of a few percent (for example Q2 between 1 day and 3 hours) are within run-to-run variation.
- **Warm-cache labelling.** Timings follow a warm-up and are warm-cache results. The one plan shown (shared read=8647) was taken on a colder run, so it is not representative of the timed runs.
- **Cache state affects the compression comparison.** The 4.3 baseline medians come from an earlier session than the compressed medians; a same-session spot check shows the baseline can be much faster, so the percentage improvements are an upper bound.
- **UTC chunk alignment.** Chunks follow UTC midnight, so a CAT day spans two chunks and 28 days produce 29 chunks. Dashboards and daily aggregates must use an explicit CAT time zone argument to agree with the CAT-day totals.
- **Missing and delayed data.** Observed totals understate true consumption by about 1% (missing readings). Late readings (2%) can be absent from reports issued before they arrive. All forecast targets are observed sums.
- **Synthetic data.** The data comes from a generator with known structure (fixed voltage range, daily and weekly shapes, injected events). Results such as the anomaly rule's failure and the model's accuracy may not carry over to real meters.

- **Anomaly evaluation.** The labelled events number only 40 and all last 12 intervals, so detection rates are coarse (each event is 2.5 percentage points).
- **Forecast scope.** The model uses only calendar features and two lags, is trained on 12 days and evaluated on 7, so seasonal behaviour beyond one month is untested. Peak hours are underpredicted in nine of ten regions.
- **Evidence not captured.** The host specification and PostgreSQL parameter values (1.1), the subscriber and stored counts of the pilot (1.3), execution plans for the 3-hour and 1-week tables (3.2), the numerical dashboard comparison and per-panel timings (7.3, 7.4) and a size measurement of the aggregate views (6.1) are not supported by captured evidence in this report.

Appendix E: glossary

Term	Meaning in this report
CAT	Central Africa Time, UTC+02:00, used for all timestamps and day boundaries.
Hypertable	A TimescaleDB table that is split automatically into chunks by time.
Chunk	One time-range partition of a hypertable, stored as its own table.
Chunk exclusion	The planner skipping chunks whose time range cannot satisfy the query filter.
Columnstore compression	Storing batches of rows as compressed column arrays to save space.
orderby / segmentby	The sort order inside a compressed batch, and the column kept uncompressed so each batch belongs to one value.
Continuous aggregate	A materialized view over a hypertable that TimescaleDB refreshes incrementally.
materialized_only	When true the view returns only stored buckets; when false it adds live raw rows newer than the watermark.
Watermark	The end of the latest materialized bucket of a continuous aggregate.
QoS 1	MQTT delivery level that guarantees at-least-once delivery, so duplicates are possible.
kW and kWh	Power (rate of demand) and energy (power over time); energy_kwh ≈ power_kw × 0.25 h per reading.
Completeness	100 × observed unique (meter, time) pairs / planned pairs.
MAE and RMSE	Mean absolute error and root mean squared error of predicted minus actual, in kWh.
Forecast origin	The time (00:00 CAT) at which a day's 24 forecasts are made; features must be available before it.
Persistence baseline	A forecast that repeats the previous day's value for the same hour.

Appendix F: index of figures

Every figure in the report, with the clock time visible on the screenshot where one was taken.

No.	Figure
1	Figure 1.1-a: EMQX 5.3.0 and TimescaleDB containers running (docker ps), 8:46 PM, 15 October 2026
2	Figure 1.2-a: CREATE TABLE for energy_live and energy_readings, create_hypertable with a 1-day chunk, and CREATE TABLE meters (8:49 PM, 15 October 2026)
3	Figure 1.2-b: populate_meters.py output with the validation check "meters table row count is: 4000" (8:57 PM, 15 October 2026)
4	Figure 1.3-a: publisher run with broker acknowledgements every 200 messages up to 2,000 (9:08 PM, 15 October 2026)
5	Figure 2.1-a: loader start, target of 10,696,848 records and progress at 100,000-row steps (9:13 PM, 15 October 2026)
6	Figure 2.1-b: load summary, 10,696,848 iterated, 10,696,848 stored, 197.52 s (9:17 PM, 15 October 2026)
7	Figure 2.2-a: diagnostic metrics, percentages, formula checks and top five meters by missing readings (9:29 PM, 15 October 2026)
8	Figure 2.3-a: cleaning completion with before and after row counts and energy sums (9:34 PM, 15 October 2026)
9	Figure 3.1-a: load of the two new hypertables and the structural audit with matching data checks (9:45 PM, 15 October 2026)
10	Figure 3.2-a: Q1 hourly average power on 21 September, 24 rows (9:58 PM, 15 October 2026)
11	Figure 3.2-b: Q2 ten highest 15-minute demand intervals (10:01 PM, 15 October 2026)
12	Figure 3.2-c: Q3 energy per meter (first 30 of 4,000 rows shown; 10:03 PM, 15 October 2026)
13	Figure 3.2-d: Q4 meter 1000000000, 22 to 28 September (10:06 PM, 15 October 2026)
14	Figure 3.2-e: Q5 full scan (10:07 PM, 15 October 2026)
15	Figure 3.2-f: timing comparison matrix and 296.36 s session runtime (10:17 PM, 15 October 2026)
16	Figure 3.2-g: EXPLAIN (ANALYZE, BUFFERS) of Q1 on energy_readings (10:11 PM, 15 October 2026)
17	Figure 4.1-a: per-chunk compression durations for both strategies and the cross-table data checks (10:31 PM, 15 October 2026)
18	Figure 4.1-b: compression policy jobs 1000 and 1001 created and paused (scheduled = f), 10:25 PM, 15 October 2026
19	Figure 4.2-a: hypertable_detailed_size for the three tables
20	Figure 4.2-b: storage, ratios, chunk tally and compression cost summary (10:46 PM, 15 October 2026)
21	Figure 4.3-a: five-configuration median latency matrix with percentage improvement (10:58 PM, 15 October 2026)
22	Figure 4.3-b: same-session spot check of Q1 on the three tables (10:36 PM, 15 October 2026)
23	Figure 4.3-c: manual timing of Q1 on the uncompressed table with \timing (10:44 PM, 15 October 2026)
24	Figure 4.3-d: compression policies resumed, compress_after 48:00:00, scheduled = t (10:53 PM, 15 October 2026)
25	Figure 5.1-a: weekly totals and three highest demand intervals (11:12 PM, 15 October 2026)
26	Figure 5.1-b: daily network energy per CAT day (top) and the three highest 15-minute demands (bottom)
27	Figure 5.2-a: totals, meter counts and energy per meter by region and by customer type (11:23 PM, 15 October 2026)
28	Figure 5.3-a: completeness by region and the two 21 September daily totals (11:37 PM, 15 October 2026)
29	Figure 5.4-a: executed detection with thresholds, detected and missed events and false-flag measures (11:54 PM, 15 October 2026)
30	Figure 5.4-b: sample flagged readings (meter_id, event_time), all from meter 1000000051 (11:57 PM, 15 October 2026)
31	Figure 6.1-a: creation of the three views (12:14 AM, 16 October 2026)
32	Figure 6.1-b: manual refresh durations of 43.69 s, 24.19 s and 3.90 s (12:15 AM, 16 October 2026)
33	Figure 6.1-c: materialized row counts, policies created and background job status (12:16 AM, 16 October 2026)
34	Figure 6.2-a: freshness transition matrix and raw table check, 3 rows and 3.5 kWh (12:44 AM, 16 October 2026)
35	Figure 7.1-a: panel 1, hourly regional energy (kWh), Raw dashboard (2:04 AM, 16 October 2026)
36	Figure 7.1-b: panel 2, daily network energy, Raw dashboard (2:06 AM, 16 October 2026)
37	Figure 7.1-c: panel 3, network energy in the four seven-day periods, Raw dashboard (2:07 AM, 16 October 2026)
38	Figure 7.1-d: panel 4, daily energy for meter 1000000000, Raw dashboard (2:08 AM, 16 October 2026)
39	Figure 7.1-e: panel 5, telemetry completeness by region (%), Raw dashboard (2:09 AM, 16 October 2026)
40	Figure 7.2-a: panel 1, hourly regional energy, Aggregate dashboard (2:13 AM, 16 October 2026)
41	Figure 7.2-b: panel 2, daily network energy, Aggregate dashboard (2:20 AM, 16 October 2026)

No.	Figure
42	Figure 7.2-c: panel 3, seven-day periods, Aggregate dashboard (2:24 AM, 16 October 2026)
43	Figure 7.2-d: panel 4, daily energy for meter 1000000000, Aggregate dashboard (2:35 AM, 16 October 2026)
44	Figure 7.2-e: panel 5, completeness by region from stored reading counts, Aggregate dashboard (2:44 AM, 16 October 2026)
45	Figure 8.1-a: dataset preparation, split sizes, features, leakage check and sample rows (3:36 AM, 16 October 2026)
46	Figure 8.2-a: baseline run with prediction count, metrics and sample predictions (3:45 AM, 16 October 2026)
47	Figure 8.3-a: validation comparison, selected configuration, test metrics and sample predictions (3:57 AM, 16 October 2026)
48	Figure 8.4-a: global and regional metrics and peak-hour errors (4:02 AM, 16 October 2026)
49	Figure 8.4-b: actual, baseline and model consumption for Region 1 over 22 to 28 September (the plot title reads "Validation Horizon" but it covers the test days)